PYTHON | CHAPTER 02

Variables
The Complete Deep Dive

Every topic of Chapter 2, taught step by step: what it is, a runnable example, the real output, then every variation and consequence, followed by graded questions with a verified answer key.

Topics	**92**	Verified demos	**153**
Sections	**8**	Graded questions	**325**

Edition of 31 October 2026. Every output shown in this book was produced by running the code on Python 3.12.3. Sizes and timings can differ on other versions; the concepts do not.

Contents

How to Use This Book 5

Section 2.1 Concept of Variables 6

2.1.1 What is a variable (name binding, not a box) . . . 6
2.1.2 Variable as reference to an object . . . 6
2.1.3 id() and type(): first tools . . . 7
2.1.4 Names vs values . . . 8
2.1.5 Dynamic typing (contrast with static typing) . . . 9
2.1.6 Strong typing (Python is strongly + dynamically typed) . . . 10
2.1.7 Objects: identity, type, value . . . 10
2.1.8 Everything is an object: first proof . . . 11
2.1.9 Variables are not declared . . . 11
2.1.10 Multiple names for the same object . . . 12
2.1.11 Rebinding does not affect other names . . . 13
2.1.12 Variable lifetime and reference counting intro . . . 13
2.1.13 Mental-model exercises (mini-lab) . . . 14

Section 2.2 Naming 16

2.2.1 Valid identifier rules . . . 16
2.2.2 Case sensitivity . . . 16
2.2.3 snake_case convention (PEP 8) . . . 17
2.2.4 Constant naming (UPPER_CASE, Final) . . . 17
2.2.5 Reserved keywords as names (error) . . . 18
2.2.6 Dunder names (__x__): intro . . . 18
2.2.7 Private-ish naming (_x, __x, __x__) . . . 19
2.2.8 Beginner pitfalls: shadowing built-ins (list, id, input) . . . 20
2.2.9 Naming for readability . . . 20
2.2.10 Throwaway name _ . . . 21
2.2.11 Unicode names and normalisation . . . 21
2.2.12 Naming in competitive programming . . . 22

Section 2.3 Assignment 23

2.3.1 Simple assignment . . . 23
2.3.2 Multiple assignment: a = b = c = 0 . . . 23
2.3.3 Tuple unpacking: a, b = 1, 2 . . . 24
2.3.4 Swap idiom: a, b = b, a . . . 25
2.3.5 Starred unpacking: a, *b = [1,2,3] . . . 25
2.3.6 Augmented assignment (+=, -=, *=, //=, **=, &=, |=, ^=, >>=, <<=, @=) . . . 26
2.3.7 Chained comparison: 1 < x < 10 . . . 27
2.3.8 Reassignment vs mutation . . . 28
2.3.9 del keyword . . . 29

2.3.10 None as default . . . 30
2.3.11 Nested unpacking . . . 30
2.3.12 Unpacking in function calls and literals . . . 31
2.3.13 Assignment targets that are not names . . . 31
2.3.14 Walrus operator := first look . . . 32
2.3.15 Annotated assignment x: int = 5 . . . 33
2.3.16 Evaluation order puzzles . . . 33
2.3.17 Assignment inside loops and conditionals . . . 34
2.3.18 Assignment mini-lab . . . 35

Section 2.4 Scope (Variables Lifespan) 37

2.4.1 Local / Enclosing / Global / Built-in (LEGB) . . . 37
2.4.2 global keyword . . . 38
2.4.3 nonlocal keyword . . . 39
2.4.4 Shadowing . . . 40
2.4.5 UnboundLocalError pitfall . . . 40
2.4.6 Module-level variables . . . 41
2.4.7 __main__ scope (intro) . . . 42
2.4.8 Block scope does not exist . . . 42
2.4.9 Comprehension scope and the loop variable leak . . . 43
2.4.10 Closures: first look . . . 43
2.4.11 Class scope . . . 44
2.4.12 Exception variable scope . . . 45
2.4.13 Namespaces and dir()/vars()/globals()/locals() . . . 45
2.4.14 Global state in competitive programming . . . 46
2.4.15 Scope mini-lab . . . 47

Section 2.5 Mutability and Identity 49

2.5.1 Mutable vs immutable (contrast table) . . . 49
2.5.2 id() behaviour . . . 50
2.5.3 is vs == (contrast, big pitfall) . . . 50
2.5.4 Small integer interning (-5..256) . . . 51
2.5.5 String interning . . . 52
2.5.6 Copy pitfalls (a = b for lists) . . . 53
2.5.7 Shallow vs deep copy (preview) . . . 54
2.5.8 Hashability . . . 54
2.5.9 Immutability is shallow . . . 55
2.5.10 Mutation through function parameters . . . 56
2.5.11 Defensive copying . . . 56
2.5.12 Immutable tricks and performance . . . 57
2.5.13 Object identity in containers . . . 58
2.5.14 Mutability mini-lab . . . 58

Section 2.6 Memory Model Internals 60

2.6.1 Objects on the heap, names in namespaces . . . 60
2.6.2 Reference counting . . . 60
2.6.3 Cyclic garbage collector . . . 61
2.6.4 Object header and size of basic objects . . . 61
2.6.5 Integer storage in CPython . . . 62
2.6.6 id reuse after deletion . . . 63
2.6.7 Weak references and del . . . 63
2.6.8 Memory profiling tools . . . 64

Section 2.7 Annotations, Constants and Typing for Variables 65

2.7.1 Variable annotations . . . 65
2.7.2 typing.Final and constants . . . 65
2.7.3 Type aliases . . . 66
2.7.4 Optional, Union and the | syntax . . . 66
2.7.5 Runtime vs static typing . . . 66
2.7.6 Mini-lab: annotate a script . . . 67

Section 2.8 Inspecting Variables 68

2.8.1 type, isinstance, id, dir, vars, repr . . . 68
2.8.2 globals() and locals() as dicts . . . 68
2.8.3 inspect module first look . . . 69
2.8.4 f"{x=}" debugging . . . 69
2.8.5 Trace tools: pdb, sys.settrace, tracemalloc . . . 70
2.8.6 Chapter 2 mini-lab: Variable Playground . . . 70

Chapter 2 Summary: The 30 Rules 72

Error Message Decoder . . . 72

Final Exam (45 mixed HARD and TRAP questions) 74

Answer Key (outputs from running the code) 78

Final exam answers . . . 88

How to Use This Book

Each topic follows the same teaching order, matching your index: **what it is**, then **a runnable example**, then **the verified output**, then **every variation and consequence**, then **graded practice**.
Green-edged boxes are code, blue-edged boxes are the exact output the code printed.

- Read the explanation, then try each example yourself before looking at the printed output.
- In the Practice list, cover the answer key at the end of the book, write your prediction, and only then run the question.
- Levels: BASIC (direct), MID (needs the model), HARD (combines ideas), TRAP (looks easy, hides a pitfall).
- Addresses and ids vary between runs, so examples compare ids instead of printing them.
- If an output surprises you, that is the point: stop and redraw the name-to-object arrows.

Core idea of the whole chapter

```
name  --->  object

assignment (x = ...)  moves an arrow
mutation   (x.append) changes the object everyone points at
'is'  asks: same object?     '=='  asks: equal values?
```

Section 2.1 Concept of Variables

2.1.1 What is a variable (name binding, not a box)

A variable in Python is a NAME that is bound to an object. The object lives somewhere in memory; the name is just a label that points to it. This is different from the "box" picture used in C, where a variable is a fixed memory slot holding a value.

Step 1: Python evaluates the right-hand side and produces (or finds) an object. Step 2: it binds the name on the left to that object in the current namespace. Assignment never copies the object.

Key rule: Rule: "x = expr" means "make the name x refer to the object that expr produced".

Example 2.1.1.1: Binding a name, then rebinding it to an object of another type

```
x = 10
print(x, type(x))
x = "ten"
print(x, type(x))
```

```
10 <class 'int'>
ten <class 'str'>
```

Example 2.1.1.2: Two names, one object: the box model cannot explain this

```
a = [1, 2]
b = a
b.append(3)
print(a, b, a is b)
```

```
[1, 2, 3] [1, 2, 3] True
```

Example 2.1.1.3: Rebinding one name leaves the other untouched

```
x = 5
y = x
x = 7
print(x, y)
```

```
7 5
```

Watch out: The "box" model predicts that b = a copies the list into a second box. It does not; both names point at the same list.

Practice 2.1.1

2.1.1-Q1 BASIC

```
x = 5; y = x; x = 7; print(y)
```

2.1.1-Q2 BASIC

```
a = [1]; b = a; b.append(2); print(a)
```

2.1.1-Q3 MID

```
a = [1]; b = a; b = b + [2]; print(a, b)
```

2.1.1-Q4 HARD

```
a = [1]; b = a; b += [2]; print(a, b)
```

2.1.1-Q5 TRAP

```
a = "hi"; b = a; b += "!"; print(a, b)
```

2.1.2 Variable as reference to an object

Think of every name as an arrow from the name to an object. Assignment moves or creates an arrow. Objects do not know which names point at them (they only keep a count). Containers also hold arrows: a list slot is an arrow to an object.

Draw it: after a = [1, 2]; b = a there is ONE list object and TWO arrows into it. After b = [1, 2] there are TWO list objects, each with one arrow.

Example 2.1.2.1: Same object versus equal objects

```
a = [1, 2]
b = a
c = [1, 2]
print(a == b, a is b)
print(a == c, a is c)
```

```
True True
True False
```

Example 2.1.2.2: A list holds references, not copies

```
x = [0]
row = [x, x]
x.append(1)
print(row)
```

```
[[0, 1], [0, 1]]
```

Example 2.1.2.3: The same arrow idea for strings and ints (immutable, but still references)

```
s = "python"
t = s
print(s is t)
t = t.upper()
print(s, t, s is t)
```

```
True
python PYTHON False
```

Practice 2.1.2

2.1.2-Q1 BASIC

```
a = [1]; c = [1]; print(a == c, a is c)
```

2.1.2-Q2 MID

```
x = [0]; row = [x, x]; row[0].append(9); print(row)
```

2.1.2-Q3 MID

```
x = [0]; row = [x[:], x[:]]; row[0].append(9); print(row)
```

2.1.2-Q4 HARD

```
a = [1]; b = [a, a]; a = [2]; print(b)
```

2.1.3 id() and type(): first tools

type(obj) tells you which class the object belongs to. id(obj) returns an integer that identifies the object during its lifetime; in CPython it is the memory address. Two names refer to the same object exactly when their ids are equal, which is what the "is" operator tests.

Never print raw ids in notes and expect them to repeat: they differ between runs and machines. Compare them instead.

Example 2.1.3.1: type() and id() comparisons (we print comparisons, not raw addresses)

```
a = 5
b = a
c = [1]
d = c
print(type(a), type(c))
print(id(a) == id(b), id(c) == id(d), id(a) == id(c))
print(a is b, c is d)
```

```
<class 'int'> <class 'list'>
True True False
True True
```

Example 2.1.3.2: type of a name does not exist; only the object has a type

```
x = 3
print(type(x))
x = 3.0
print(type(x))
```

```
<class 'int'>
<class 'float'>
```

Practice 2.1.3

2.1.3-Q1 BASIC

```
a = [1]; b = a; print(id(a) == id(b))
```

2.1.3-Q2 BASIC

```
print(type(5), type(5.0), type("5"), type([5]))
```

2.1.3-Q3 MID

```
a = [1]; b = list(a); print(a == b, id(a) == id(b))
```

2.1.3-Q4 HARD

```
print(type(type(5)))
```

2.1.4 Names vs values

A value is the content of an object (5, "hi", [1, 2]). A name is a label in a namespace. Functions receive references to the objects passed in ("call by object reference"), so the function parameter is a new name for the same object.

Example 2.1.4.1: A parameter is another name for the caller's object

```
def show(p):
    print(p is data)
data = [1, 2]
show(data)
```

```
True
```

Example 2.1.4.2: Reassigning the parameter does not affect the caller; mutating does

```
def f(p):
    p = p + [99]
def g(p):
    p.append(99)
data = [1]
f(data); print(data)
g(data); print(data)
```

```
[1]
[1, 99]
```

Practice 2.1.4

2.1.4-Q1 BASIC

```
def f(n): n += 1
x = 1; f(x); print(x)
```

2.1.4-Q2 MID

```
def f(l): l += [1]
x = []; f(x); print(x)
```

2.1.4-Q3 MID

```
def f(l): l = l + [1]
x = []; f(x); print(x)
```

2.1.4-Q4 HARD

```
def f(l): l[:] = [7]
x = [1, 2]; f(x); print(x)
```

2.1.5 Dynamic typing (contrast with static typing)

In a statically typed language (C, Java) the variable has a declared type and the compiler rejects wrong assignments. In Python the TYPE BELONGS TO THE OBJECT; a name can be rebound to any object at any time. Type errors appear at run time, when an operation meets an unsupported object.

Optional type hints (x: int) are read by tools like mypy, not enforced by the interpreter.

Example 2.1.5.1: One name, three types

```
v = 1
print(type(v).__name__)
v = "one"
print(type(v).__name__)
v = [1]
print(type(v).__name__)
```

```
int
str
list
```

Example 2.1.5.2: Hints are not enforced

```
x: int = "hello"
print(x, type(x).__name__)
print(__annotations__)
```

```
hello str
{'x': <class 'int'>}
```

Example 2.1.5.3: Dynamic typing errors surface at the operation

```
def add(a, b):
    return a + b
print(add(2, 3))
print(add("2", "3"))
print(add(2, "3"))
```

```
5
23
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Practice 2.1.5

2.1.5-Q1 BASIC

```
x = 1; x = "a"; print(x)
```

2.1.5-Q2 MID

```
def f(a, b): return a * b
print(f(3, "ab"), f("ab", 3))
```

2.1.5-Q3 MID

```
x: int = 2.5; print(x)
```

2.1.5-Q4 HARD

```
def f(a, b): return a + b
print(f([1], [2]), f((1,), (2,)))
```

2.1.6 Strong typing (Python is strongly + dynamically typed)

Strong typing means the language does not silently convert between unrelated types. "1" + 1 raises TypeError in Python; JavaScript would return "11". Python does promote numbers (int to float to complex) because they form one numeric family.

Example 2.1.6.1: No silent coercion between str and int

```
try:
    print("1" + 1)
except TypeError as e:
    print("TypeError:", e)
print("1" * 3)
print(1 + 2.5, 1 + 2j, True + 1)
```

```
TypeError: can only concatenate str (not "int") to str
111
3.5 (1+2j) 2
```

Example 2.1.6.2: Conversion must be explicit

```
print(int("1") + 1)
print("1" + str(1))
```

```
2
11
```

Practice 2.1.6

2.1.6-Q1 BASIC

```
print("a" * 3)
```

2.1.6-Q2 MID

```
print(1 + True)
```

2.1.6-Q3 MID

```
print(2 * "ab" + "c")
```

2.1.6-Q4 HARD

```
print("5" + 5)
```

2.1.7 Objects: identity, type, value

Every object has three attributes: IDENTITY (never changes, id()), TYPE (never changes, type()), VALUE (can change only if the object is mutable). Names have none of these; they only point.

Example 2.1.7.1: Mutating changes the value, not identity

```
l = [1]
i0 = id(l)
l.append(2)
print(l, id(l) == i0)
```

```
[1, 2] True
```

Example 2.1.7.2: Immutable objects: "changing" makes a new object

```
s = "ab"
i0 = id(s)
s += "c"
print(s, id(s) == i0)
```

```
abc False
```

Practice 2.1.7

2.1.7-Q1 BASIC

```
l = [1]; i = id(l); l.append(2); print(id(l) == i)
```

2.1.7-Q2 MID

```
t = (1, 2); i = id(t); t += (3,); print(t, id(t) == i)
```

2.1.7-Q3 HARD

```
x = 5; i = id(x); x += 1; print(id(x) == i)
```

2.1.8 Everything is an object: first proof

Numbers, strings, functions, classes, modules and even types are objects with attributes and methods. This is why you can pass a function as an argument or call a method on a literal.

Example 2.1.8.1: Methods on literals, functions as objects

```
print((5).bit_length(), "abc".upper())
print(type(len), type(type), type(int))
f = len
print(f([1, 2, 3]))
print(hasattr(5, "__add__"), isinstance(int, object))
```

```
3 ABC
<class 'builtin_function_or_method'> <class 'type'> <class 'type'>
3
True True
```

Practice 2.1.8

2.1.8-Q1 BASIC

```
print((255).bit_length())
```

2.1.8-Q2 MID

```
f = str.upper; print(f("hi"))
```

2.1.8-Q3 MID

```
print(type(print).__name__)
```

2.1.8-Q4 HARD

```
print(isinstance(int, type), isinstance(type, object), isinstance(object, type))
```

2.1.9 Variables are not declared

There is no declaration statement. The first assignment creates the name. Using a name that was never bound raises NameError. Deleting a name (del) removes the binding.

Example 2.1.9.1: NameError and creation by assignment

```
try:
    print(y)
except NameError as e:
    print("NameError:", e)
y = 1
print(y)
del y
try:
    print(y)
except NameError as e:
    print("NameError:", e)
```

```
NameError: name 'y' is not defined
1
NameError: name 'y' is not defined
```

Practice 2.1.9

2.1.9-Q1 BASIC

```
x = 1; del x; print("x" in dir())
```

2.1.9-Q2 MID

```
if False:
    z = 1
try:
    print(z)
except NameError as e:
    print(type(e).__name__)
```

2.1.9-Q3 HARD

```
try:
    print(w)
except NameError:
    print("no w")
w = 3
print(w)
```

2.1.10 Multiple names for the same object

Aliasing means two or more names point to one object. For immutable objects it is invisible. For mutable objects it is the source of many bugs because a change through one name appears through all.

Example 2.1.10.1: a = b = [] shares one list

```
a = b = []
a.append(1)
print(b, a is b)
```

```
[1] True
```

Example 2.1.10.2: Independent lists need independent objects

```
a = []
b = []
a.append(1)
print(b, a is b)
```

```
[] False
```

Example 2.1.10.3: Detecting aliasing with is

```
x = [1, 2]
y = x
z = x.copy()
print(x is y, x is z, x == z)
```

```
True False True
```

Practice 2.1.10

2.1.10-Q1 BASIC

```
a = b = []; a.append(1); print(b)
```

2.1.10-Q2 MID

```
a = b = 0; a += 1; print(a, b)
```

2.1.10-Q3 MID

```
a, b = [], []; a.append(1); print(b)
```

2.1.10-Q4 HARD

```
a = b = [[0]]; a[0].append(1); print(b)
```

2.1.10-Q5 TRAP

```
a = b = [1]; a = a + [2]; print(b)
```

2.1.11 Rebinding does not affect other names

Rebinding (x = ...) moves one arrow. It never changes other names that pointed at the old object. Mutation (x.append) changes the object everyone points at. Always ask: am I moving an arrow or changing the object?

Example 2.1.11.1: Rebind vs mutate, side by side

```
a = [1]; b = a
a = [2]
print("after rebind:", a, b)
a = [1]; b = a
a.append(2)
print("after mutate:", a, b)
```

```
after rebind: [2] [1]
after mutate: [1, 2] [1, 2]
```

Practice 2.1.11

2.1.11-Q1 BASIC

```
a = 1; b = a; a = 2; print(b)
```

2.1.11-Q2 MID

```
a = [1]; b = a; a = a + [2]; print(b)
```

2.1.11-Q3 MID

```
a = [1]; b = a; a.extend([2]); print(b)
```

2.1.11-Q4 HARD

```
a = [1]; b = a; a *= 2; print(b)
```

2.1.12 Variable lifetime and reference counting intro

CPython counts how many references each object has. When the count reaches zero the object is freed immediately. del removes a NAME, not the object; the object disappears only when no references remain. sys.getrefcount(x) reports one extra reference because passing x as an argument creates a temporary one.

Example 2.1.12.1: Observing reference counts (differences, not absolutes)

```
import sys
x = []
base = sys.getrefcount(x)
y = x
print(sys.getrefcount(x) - base)
z = [x, x]
print(sys.getrefcount(x) - base)
del y, z
print(sys.getrefcount(x) - base)
```

```
1
3
0
```

Example 2.1.12.2: A weak reference shows when the object really dies

```
import weakref
class Box: pass
b = Box()
w = weakref.ref(b)
c = b
del b
print(w() is not None)
del c
print(w() is None)
```

```
True
True
```

Practice 2.1.12

2.1.12-Q1 BASIC

```
import sys; x = []; r = sys.getrefcount(x); y = x; print(sys.getrefcount(x) - r)
```

2.1.12-Q2 MID

```
x = [1]; y = x; del x; print(y)
```

2.1.12-Q3 HARD

```
import weakref
class B: pass
b = B(); w = weakref.ref(b); del b; print(w() is None)
```

2.1.13 Mental-model exercises (mini-lab)

Trace each program by drawing arrows before reading the output. Every answer below was produced by running the code.

Practice 2.1.13

2.1.13-Q1 BASIC

```
a = 3; b = a; a = 4; print(a, b)
```

2.1.13-Q2 BASIC

```
a = [1]; b = a; b.append(2); print(a, b)
```

2.1.13-Q3 MID

```
a = [1, 2]; b = a[:]; b.append(3); print(a, b)
```

2.1.13-Q4 MID

```
a = (1, [2]); b = a; b[1].append(3); print(a)
```

2.1.13-Q5 MID

```
a = "x"; b = a; a += "y"; print(a, b)
```

2.1.13-Q6 HARD

```
a = [1]; b = [a] * 3; a.append(2); print(b)
```

2.1.13-Q7 HARD

```
a = [1]; b = [a[:]] * 3; b[0].append(2); print(b)
```

2.1.13-Q8 HARD

```
x = [1]; y = x; x = x + [2]; y += [3]; print(x, y)
```

2.1.13-Q9 TRAP

```
a = b = c = [0]; b = b + [1]; c += [2]; print(a, b, c)
```

2.1.13-Q10 TRAP

```
a = [1]; b = a; a, b = b, a + [9]; print(a, b)
```

Section 2.2 Naming

2.2.1 Valid identifier rules

An identifier may contain letters, digits and underscore, must not start with a digit, and must not be a keyword. Unicode letters are allowed. str.isidentifier() checks syntax (not keywords); keyword.iskeyword() checks reserved words.

Example 2.2.1.1: Testing candidate names

```
import keyword
for s in ["name", "_x", "x1", "1x", "my-var", "class", "naïve", "π"]:
    print(f"{s!r:10} identifier={s.isidentifier()} keyword={keyword.iskeyword(s)}")
```

```
'name'     identifier=True keyword=False
'_x'       identifier=True keyword=False
'x1'       identifier=True keyword=False
'1x'       identifier=False keyword=False
'my-var'   identifier=False keyword=False
'class'    identifier=True keyword=True
'naïve'    identifier=True keyword=False
'π'        identifier=True keyword=False
```

Practice 2.2.1

2.2.1-Q1 BASIC

```
print("var_1".isidentifier(), "1var".isidentifier())
```

2.2.1-Q2 MID

```
import keyword; print(keyword.iskeyword("None"), keyword.iskeyword("print"))
```

2.2.1-Q3 MID

```
print("a b".isidentifier(), "a_b".isidentifier())
```

2.2.1-Q4 HARD

```
import keyword; print(keyword.iskeyword("match"), "match".isidentifier())
```

2.2.2 Case sensitivity

age, Age and AGE are three different names. Typos in case create new names silently when you assign, and NameError when you read.

Example 2.2.2.1: Three different names

```
age = 1; Age = 2; AGE = 3
print(age, Age, AGE)
try:
    print(agE)
except NameError as e:
    print("NameError:", e)
```

```
1 2 3
NameError: name 'agE' is not defined
```

Practice 2.2.2

2.2.2-Q1 BASIC

```
a = 1; A = 2; print(a + A)
```

2.2.2-Q2 MID

```
total = 5
Total = total + 1
print(total, Total)
```

2.2.2-Q3 HARD

```
x = 1
X = x
x = 2
print(X)
```

2.2.3 snake_case convention (PEP 8)

PEP 8: variables, functions and modules use lower_snake_case; classes use CapWords; constants use UPPER_SNAKE_CASE. Conventions are enforced by linters, not the interpreter.

Example 2.2.3.1: Style table demonstrated

```
user_name = "ana"
MAX_RETRIES = 3
class HttpClient: pass
print(user_name, MAX_RETRIES, HttpClient.__name__)
```

```
ana 3 HttpClient
```

Practice 2.2.3

2.2.3-Q1 BASIC

```
print("snake_case".islower(), "SnakeCase".islower())
```

2.2.3-Q2 MID

```
print("MAX_SIZE".isupper())
```

2.2.3-Q3 HARD

```
print("CapWords".isidentifier(), "CapWords"[0].isupper())
```

2.2.4 Constant naming (UPPER_CASE, Final)

Python has no real constants. UPPER_CASE is a promise to other programmers. typing.Final lets a type checker reject reassignment; at run time nothing stops it.

Example 2.2.4.1: Final is checked by tools only

```
from typing import Final
RATE: Final = 0.2
RATE = 0.5
print(RATE)
```

```
0.5
```

Practice 2.2.4

2.2.4-Q1 BASIC

```
PI = 3.14; PI = 3; print(PI)
```

2.2.4-Q2 MID

```
from typing import Final
X: Final = 1
X += 1
print(X)
```

2.2.4-Q3 HARD

```
import math; math.pi = 3; print(math.pi)
```

2.2.5 Reserved keywords as names (error)

Hard keywords (class, def, if, for, None, True, ...) cannot be names. Soft keywords (match, case, type, _) can still be names but should not. A trailing underscore (class_) is the standard workaround.

Example 2.2.5.1: What happens

```
import keyword
print(len(keyword.kwlist), keyword.softkwlist)
try:
    compile("class = 1", "<s>", "exec")
except SyntaxError as e:
    print("SyntaxError:", e.msg)
class_ = 1
print(class_)
match = 5
print(match)
```

```
35 ['_', 'case', 'match', 'type']
SyntaxError: invalid syntax
1
5
```

Practice 2.2.5

2.2.5-Q1 BASIC

```
import keyword; print("def" in keyword.kwlist)
```

2.2.5-Q2 MID

```
import keyword; print("print" in keyword.kwlist)
```

2.2.5-Q3 MID

```
try:
    compile("None = 1", "<s>", "exec")
except SyntaxError as e:
    print("SyntaxError")
```

2.2.5-Q4 HARD

```
import keyword; print("_" in keyword.softkwlist, "match" in keyword.softkwlist)
```

2.2.6 Dunder names (__x__): intro

Names that begin and end with double underscores belong to Python (__name__, __doc__, __init__, __add__). Use them to hook into language behaviour, but never invent your own.

Example 2.2.6.1: Built-in dunder attributes

```
def f():
    "doc text"
print(f.__name__, f.__doc__)
print(__name__)
print((5).__add__(3), (5).__class__)
```

```
f doc text
__main__
8 <class 'int'>
```

Practice 2.2.6

2.2.6-Q1 BASIC

```
print((2).__mul__(4))
```

2.2.6-Q2 MID

```
print("abc".__len__())
```

2.2.6-Q3 HARD

```
print(5 .__eq__(5.0), 5 .__eq__("5"))
```

2.2.7 Private-ish naming (_x, __x, __x__)

_x: "internal use" hint. __x inside a class: name mangling to _ClassName__x, which avoids accidental clashes in subclasses. __x__: reserved dunder. None of these are true privacy.

Example 2.2.7.1: Mangling in action

```
class A:
    def __init__(self):
        self.pub = 1
        self._prot = 2
        self.__priv = 3
a = A()
print(a.pub, a._prot)
print([n for n in vars(a)])
print(a._A__priv)
try:
    print(a.__priv)
except AttributeError as e:
    print("AttributeError:", e)
```

```
1 2
['pub', '_prot', '_A__priv']
3
AttributeError: 'A' object has no attribute '__priv'
```

Practice 2.2.7

2.2.7-Q1 BASIC

```
class A:
    _x = 1
print(A._x)
```

2.2.7-Q2 MID

```
class A:
    def __init__(self): self.__v = 1
print(sorted(vars(A())))
```

2.2.7-Q3 HARD

```
class A:
    __v = 1
class B(A): pass
print(hasattr(B, "__v"), hasattr(B, "_A__v"))
```

2.2.8 Beginner pitfalls: shadowing built-ins (list, id, input)

Assigning to a name like list, id, input, sum or max hides the built-in in that scope. Later calls fail with confusing errors. del the name to restore the built-in.

Example 2.2.8.1: The classic list bug

```
list = [1, 2, 3]
try:
    print(list("abc"))
except TypeError as e:
    print("TypeError:", e)
del list
print(list("abc"))
```

```
TypeError: 'list' object is not callable
['a', 'b', 'c']
```

Example 2.2.8.2: Safe check

```
import builtins
print(hasattr(builtins, "sum"), hasattr(builtins, "total"))
```

```
True False
```

Practice 2.2.8

2.2.8-Q1 BASIC

```
sum = 10; del sum; print(sum([1, 2]))
```

2.2.8-Q2 MID

```
max = 3
try:
    print(max(1, 2))
except TypeError:
    print("shadowed")
```

2.2.8-Q3 HARD

```
id = 5
print(type(id).__name__)
del id
print(type(id).__name__)
```

2.2.9 Naming for readability

Use descriptive names; put units in names (timeout_s); boolean names read as questions (is_ready, has_items); avoid l, O, I as single letters (look like 1 and 0). Loop variables can be short (i, k, x) when the scope is tiny.

Example 2.2.9.1: Same code, two names

```
d = 86400
total_seconds_per_day = 24 * 60 * 60
print(d == total_seconds_per_day)
```

```
True
```

Practice 2.2.9

2.2.9-Q1 BASIC

```
is_ready = True; print(not is_ready)
```

2.2.9-Q2 MID

```
timeout_s = 30; timeout_ms = timeout_s * 1000; print(timeout_ms)
```

2.2.10 Throwaway name _

_ marks "I do not care". In the REPL it holds the last result. It is also a common loop variable and unpacking target.

Example 2.2.10.1: Common uses

```
for _ in range(3):
    print("hi", end=" ")
print()
a, _, c = (1, 2, 3)
print(a, c)
first, *_ = [10, 20, 30]
print(first)
```

```
hi hi hi
1 3
10
```

Practice 2.2.10

2.2.10-Q1 BASIC

```
a, _ = (1, 2); print(a)
```

2.2.10-Q2 MID

```
_, _, z = (1, 2, 3); print(z)
```

2.2.10-Q3 HARD

```
for _ in range(2): pass
print(_)
```

2.2.11 Unicode names and normalisation

Python accepts Unicode letters in identifiers and normalises them with NFKC, so different spellings can be the same name. Lookalike characters (Latin a vs Cyrillic a) create different names, a source of bugs and attacks. Prefer ASCII.

Example 2.2.11.1: NFKC folding and lookalikes

```
import unicodedata
fi = 1
print(fi)
a1 = "latin"
a = "cyrillic"
print(a1, "a" == "\u0430", unicodedata.name("\u0430"))
```

```
1
latin False CYRILLIC SMALL LETTER A
```

Practice 2.2.11

2.2.11-Q1 BASIC

```
print("ñ".isidentifier())
```

2.2.11-Q2 MID

```
print("fi".isidentifier())
```

2.2.11-Q3 HARD

```
import unicodedata; print(unicodedata.normalize("NFKC", "fi"))
```

2.2.12 Naming in competitive programming

Short names (n, m, a, dp, ans) are accepted because functions are small and time is limited. Keep a consistent convention: n sizes, i j k indices, a b arrays, ans result. Outside contests, prefer readable names.

Example 2.2.12.1: A typical contest header

```
n = 5
a = [3, 1, 4, 1, 5]
ans = sum(a)
print(n, ans)
```

```
5 14
```

Practice 2.2.12

2.2.12-Q1 BASIC

```
n = 3; a = [1] * n; print(a)
```

2.2.12-Q2 MID

```
a = [1, 2, 3]; ans = max(a) - min(a); print(ans)
```

Section 2.3 Assignment

2.3.1 Simple assignment

"x = 10" has two steps. First the right side is fully evaluated to an object. Then the name is bound to that object. Assignment is a statement: it produces no value, so "print(x = 5)" is a syntax error (the walrus operator := is the expression form).

Example 2.3.1.1: Evaluation order

```
x = 10
y = x + 1
x = x * 2
print(x, y)
```

```
20 11
```

Example 2.3.1.2: Assignment is a statement

```
try:
    compile("if z = 5: pass", "<s>", "exec")
except SyntaxError:
    print("SyntaxError: assignment is not an expression")
print(compile("(z := 5)", "<s>", "eval") is not None)
```

```
SyntaxError: assignment is not an expression
True
```

Practice 2.3.1

2.3.1-Q1 BASIC

```
x = 2; x = x * x; print(x)
```

2.3.1-Q2 MID

```
x = 1; x = x + x; x = x + x; print(x)
```

2.3.1-Q3 HARD

```
x = 3; y = (x := x + 1) * 2; print(x, y)
```

2.3.2 Multiple assignment: a = b = c = 0

Chained assignment evaluates the right side ONCE and binds every name, left to right, to that single object. Safe for immutable values; with a mutable value all names are aliases.

Example 2.3.2.1: Immutable value is harmless

```
a = b = c = 0
a += 1
print(a, b, c)
```

```
1 0 0
```

Example 2.3.2.2: Mutable value is shared

```
a = b = c = []
a.append(1)
print(a, b, c, a is c)
```

```
[1] [1] [1] True
```

Example 2.3.2.3: Build independent objects with a comprehension or separate lines

```
a, b, c = [], [], []
a.append(1)
print(a, b, c)
```

```
[1] [] []
```

Practice 2.3.2

2.3.2-Q1 BASIC

```
a = b = 5; a = 6; print(b)
```

2.3.2-Q2 MID

```
a = b = [1]; b.append(2); print(a)
```

2.3.2-Q3 MID

```
a = b = [1]; b = b + [2]; print(a)
```

2.3.2-Q4 HARD

```
a = b = c = [0]; c += [1]; print(a, b)
```

2.3.2-Q5 TRAP

```
a = b = {"k": 1}; b["k"] = 2; print(a)
```

2.3.3 Tuple unpacking: a, b = 1, 2

The right side builds a tuple (packing), then its items are bound to the targets (unpacking). Any iterable works on the right. The count must match exactly or ValueError is raised.

Example 2.3.3.1: Basic, any iterable, wrong counts

```
a, b = 1, 2
print(a, b)
x, y, z = "abc"
print(x, y, z)
p, q = {"k1": 1, "k2": 2}
print(p, q)
try:
    a, b = [1, 2, 3]
except ValueError as e:
    print("ValueError:", e)
try:
    a, b, c = [1, 2]
except ValueError as e:
    print("ValueError:", e)
```

```
1 2
a b c
k1 k2
ValueError: too many values to unpack (expected 2)
ValueError: not enough values to unpack (expected 3, got 2)
```

Practice 2.3.3

2.3.3-Q1 BASIC

```
a, b = "xy"; print(b)
```

2.3.3-Q2 MID

```
a, b = {"u": 1, "v": 2}.values(); print(a + b)
```

2.3.3-Q3 MID

```
a, b = (1,); print(a)
```

2.3.3-Q4 HARD

```
a, b = range(2, 4); print(a * b)
```

2.3.4 Swap idiom: a, b = b, a

The right side tuple (b, a) is built first using the current values, then the names are bound. That is why no temporary variable is needed. Bytecode uses a SWAP instruction for two or three items.

Example 2.3.4.1: Swap and Fibonacci step

```
a, b = 1, 2
a, b = b, a
print(a, b)
a, b = 0, 1
for _ in range(8):
    a, b = b, a + b
print(a, b)
```

```
2 1
21 34
```

Example 2.3.4.2: Why order matters

```
a, b = 1, 2
a = b
b = a
print(a, b)
```

```
2 2
```

Example 2.3.4.3: The bytecode

```
import dis
dis.dis("a, b = b, a")
```

```
  0           0 RESUME                   0

  1           2 LOAD_NAME                0 (b)
              4 LOAD_NAME                1 (a)
              6 SWAP                     2
              8 STORE_NAME               1 (a)
             10 STORE_NAME               0 (b)
             12 RETURN_CONST             0 (None)
```

Practice 2.3.4

2.3.4-Q1 BASIC

```
a, b = 3, 4; a, b = b, a; print(a, b)
```

2.3.4-Q2 MID

```
a, b, c = 1, 2, 3; a, b, c = c, a, b; print(a, b, c)
```

2.3.4-Q3 HARD

```
a, b = 1, 2; a, b = b, a + b; print(a, b)
```

2.3.4-Q4 TRAP

```
a = 1; b = 2; a = b; b = a; print(a, b)
```

2.3.5 Starred unpacking: a, *b = [1,2,3]

One starred target collects the leftover items into a LIST (even when the source is a tuple or string). Only one star is allowed per target list. The star can sit at the start, middle or end; with no leftovers it collects [].

Example 2.3.5.1: Positions of the star

```
a, *b = [1, 2, 3]
print(a, b)
*h, t = "abcd"
print(h, t)
first, *mid, last = (1, 2, 3, 4, 5)
print(first, mid, last)
x, *rest = [1]
print(x, rest)
```

```
1 [2, 3]
['a', 'b', 'c'] d
1 [2, 3, 4] 5
1 []
```

Example 2.3.5.2: Errors

```
try:
    compile("*a, *b = [1, 2]", "<s>", "exec")
except SyntaxError as e:
    print("SyntaxError:", e.msg)
try:
    a, *b, c = [1]
except ValueError as e:
    print("ValueError:", e)
```

```
SyntaxError: multiple starred expressions in assignment
ValueError: not enough values to unpack (expected at least 2, got 1)
```

Practice 2.3.5

2.3.5-Q1 BASIC

```
a, *b = [1, 2, 3]; print(b)
```

2.3.5-Q2 MID

```
*a, b = "xyz"; print(a, b)
```

2.3.5-Q3 MID

```
a, *b = (1,); print(b)
```

2.3.5-Q4 HARD

```
a, *b, c = range(5); print(a, b, c)
```

2.3.6 Augmented assignment (+=, -=, *=, //=, **=, &=, |=, ^=, >>=, <<=, @=)

x += y is x = x + y with the target evaluated once, BUT the object may be mutated in place: Python first tries x.__iadd__(y). Mutable types (list, set, dict, bytearray) implement in-place versions; immutable types (int, str, tuple) fall back to x = x + y and rebind.

Example 2.3.6.1: Every operator

```
x = 20
x += 2; print(x)
x -= 5; print(x)
x *= 2; print(x)
x //= 3; print(x)
x **= 2; print(x)
x %= 7; print(x)
x &= 6; x |= 9; x ^= 5; print(x)
x <<= 2; x >>= 1; print(x)
```

```
22
17
34
11
121
2
14
28
```

Example 2.3.6.2: In place vs rebind

```
l = [1]; m = l
l += [2]
print(m)
t = (1,); u = t
t += (2,)
print(u, t)
```

```
[1, 2]
(1,) (1, 2)
```

Example 2.3.6.3: Matrix operator exists but no builtin type uses it

```
try:
    x = 1; x @= 2
except TypeError as e:
    print("TypeError:", e)
```

```
TypeError: unsupported operand type(s) for @=: 'int' and 'int'
```

Practice 2.3.6

2.3.6-Q1 BASIC

```
x = 5; x *= 3; x -= 1; print(x)
```

2.3.6-Q2 MID

```
l = [1]; m = l; l += [2]; print(m)
```

2.3.6-Q3 MID

```
s = {1}; t = s; s |= {2}; print(t)
```

2.3.6-Q4 HARD

```
l = [1]; m = l; l = l + [2]; print(m)
```

2.3.6-Q5 TRAP

```
t = ([1], 2)
try:
    t[0] += [9]
except TypeError as e:
    print("TypeError")
print(t)
```

2.3.7 Chained comparison: 1 < x < 10

a < b < c means (a < b) and (b < c), with b evaluated only once. Chains work with any comparison operators, including == and in.

Example 2.3.7.1: Basic and evaluation count

```
x = 5
print(1 < x < 10, 1 < x < 3)
def f():
    print("f called")
    return 5
print(1 < f() < 10)
```

```
True False
f called
True
```

Example 2.3.7.2: Surprising chains

```
print(1 < 2 > 1)
print(3 == 3 < 4)
print(1 == 1 in [1])
print(1 < 2 < 3 < 2)
```

```
True
True
True
False
```

Practice 2.3.7

2.3.7-Q1 BASIC

```
print(1 < 2 < 3)
```

2.3.7-Q2 MID

```
print(5 > 3 > 4)
```

2.3.7-Q3 MID

```
print(0 < 0 < 1 or 1)
```

2.3.7-Q4 HARD

```
print(1 == 1 == True)
```

2.3.7-Q5 HARD

```
print(False == False in [False])
```

2.3.8 Reassignment vs mutation

Reassignment: the NAME moves to a new object. Mutation: the OBJECT itself changes. Use id() or "is" to tell which happened. This is the single most important idea in the chapter.

Example 2.3.8.1: Same experiment on a list and a string

```
l = [1]; i = id(l)
l.append(2)
print("list mutated, same object:", id(l) == i)
l = l + [3]
print("list rebound, same object:", id(l) == i)
s = "ab"; j = id(s)
s += "c"
print("str rebound, same object:", id(s) == j)
```

```
list mutated, same object: True
list rebound, same object: False
str rebound, same object: False
```

Example 2.3.8.2: Which methods mutate

```
l = [3, 1, 2]
print(l.sort(), l)
print(sorted([3, 1, 2]))
print(l.reverse(), l)
```

```
None [1, 2, 3]
[1, 2, 3]
None [3, 2, 1]
```

Example 2.3.8.3: Mutation visible through every alias

```
a = [1]; b = a
a.append(2)
a = a + [3]
print(a, b)
```

```
[1, 2, 3] [1, 2]
```

Practice 2.3.8

2.3.8-Q1 BASIC

```
l = [1]; l.append(2); print(l)
```

2.3.8-Q2 MID

```
l = [3, 1]; r = l.sort(); print(r, l)
```

2.3.8-Q3 MID

```
a = [1]; b = a; a = a + [2]; print(b)
```

2.3.8-Q4 HARD

```
a = [1]; b = a; a += [2]; a = a + [3]; print(a, b)
```

2.3.8-Q5 TRAP

```
x = [1, 2]; y = x.append(3); print(y)
```

2.3.9 del keyword

del removes a binding (or a container item/slice). It does not directly destroy objects; they go away only when the reference count reaches zero. After del the name raises NameError.

Example 2.3.9.1: Forms of del

```
x = 1; del x
l = [0, 1, 2, 3, 4]
del l[0]; print(l)
del l[1:3]; print(l)
d = {"a": 1, "b": 2}
del d["a"]; print(d)
a = [1]; b = a
del a
print(b)
```

```
[1, 2, 3, 4]
[1, 4]
{'b': 2}
[1]
```

Practice 2.3.9

2.3.9-Q1 BASIC

```
l = [1, 2, 3]; del l[1]; print(l)
```

2.3.9-Q2 MID

```
l = list(range(6)); del l[::2]; print(l)
```

2.3.9-Q3 MID

```
a = [1]; b = a; del a; print(b)
```

2.3.9-Q4 HARD

```
l = [1, 2, 3]; del l[:]; print(l)
```

2.3.10 None as default

None is the standard "no value yet" marker. It is a singleton, falsy, and distinct from 0, "" and []. Test with "is None".

Example 2.3.10.1: None versus other falsy values

```
for v in [None, 0, "", []]:
    print(repr(v), bool(v), v is None)
result = None
if result is None:
    print("not computed yet")
```

```
None False True
0 False False
'' False False
[] False False
not computed yet
```

Practice 2.3.10

2.3.10-Q1 BASIC

```
print(None is None)
```

2.3.10-Q2 MID

```
x = None; print(x == 0, x is None)
```

2.3.10-Q3 HARD

```
print(None == False, not None)
```

2.3.11 Nested unpacking

Unpacking patterns can be nested to mirror the data structure. They are also used in for loops over enumerate, zip and dict.items().

Example 2.3.11.1: Nested structures

```
(a, (b, c)) = (1, (2, 3))
print(a, b, c)
for i, (k, v) in enumerate({"x": 1, "y": 2}.items()):
    print(i, k, v)
for a, b in zip("ab", [1, 2]):
    print(a, b)
[(p, q), r] = [(1, 2), 3]
print(p, q, r)
```

```
1 2 3
0 x 1
1 y 2
a 1
b 2
1 2 3
```

Practice 2.3.11

2.3.11-Q1 BASIC

```
(a, b), c = (1, 2), 3; print(a + b + c)
```

2.3.11-Q2 MID

```
for k, v in {"a": 1}.items(): print(k, v)
```

2.3.11-Q3 HARD

```
(a, *b), c = ("xyz", 9); print(a, b, c)
```

2.3.12 Unpacking in function calls and literals

* unpacks an iterable into positional arguments or list/tuple/set items; ** unpacks a dict into keyword arguments or dict items (PEP 448).

Example 2.3.12.1: Calls and literals

```
def f(a, b, c): return a + b + c
print(f(*[1, 2, 3]), f(**{"a": 1, "b": 2, "c": 3}))
print([*"ab", *[1, 2]])
print({**{"a": 1}, **{"b": 2, "a": 9}})
print({*[1, 2], 3})
```

```
6 6
['a', 'b', 1, 2]
{'a': 9, 'b': 2}
{1, 2, 3}
```

Practice 2.3.12

2.3.12-Q1 BASIC

```
print([*range(3)])
```

2.3.12-Q2 MID

```
print({**{"a": 1}, "a": 2})
```

2.3.12-Q3 HARD

```
print([*"hi", *(1, 2)], (*[1], *[2]))
```

2.3.13 Assignment targets that are not names

The left side can be a name, an item (x[0]), a slice (x[1:3]), an attribute (obj.a) or a tuple/list of those. You cannot assign to literals, function calls or expressions.

Example 2.3.13.1: Legal targets

```
l = [0, 1, 2, 3]
l[0] = 9
l[1:3] = ["a", "b", "c"]
print(l)
class O: pass
o = O(); o.v = 5
print(o.v)
d = {}
d["k"] = 1
d["k"], l[0] = 2, 100
print(d, l[0])
```

```
[9, 'a', 'b', 'c', 3]
5
{'k': 2} 100
```

Example 2.3.13.2: Illegal

```
for code in ["1 = x", "f() = 1", "x + 1 = 2"]:
    try:
        compile(code, "<s>", "exec")
    except SyntaxError as e:
        print(code, "->", e.msg)
```

```
1 = x -> cannot assign to literal here. Maybe you meant '==' instead of '='?
f() = 1 -> cannot assign to function call here. Maybe you meant '==' instead of '='?
x + 1 = 2 -> cannot assign to expression here. Maybe you meant '==' instead of '='?
```

Practice 2.3.13

2.3.13-Q1 BASIC

```
l = [1, 2, 3]; l[1:2] = [7, 8]; print(l)
```

2.3.13-Q2 MID

```
l = [1, 2, 3]; l[:] = []; print(l)
```

2.3.13-Q3 HARD

```
l = list(range(6)); l[::2] = ["a", "b", "c"]; print(l)
```

2.3.14 Walrus operator := first look

:= assigns inside an expression and gives the value. Useful in while loops and comprehensions to avoid repeating a computation. Parenthesise it in most places. The name is bound in the enclosing function scope.

Example 2.3.14.1: Where it helps

```
data = [1, 5, 8, 12]
if (n := len(data)) > 3:
    print("long list", n)
print([y for x in data if (y := x * 2) > 10])
print(y)
```

```
long list 4
[16, 24]
24
```

Example 2.3.14.2: Needs parentheses at statement level

```
try:
    compile("x := 5", "<s>", "exec")
except SyntaxError as e:
    print("SyntaxError:", e.msg)
print((x := 5))
```

```
SyntaxError: invalid syntax
5
```

Practice 2.3.14

2.3.14-Q1 BASIC

```
print((n := 4) + 1, n)
```

2.3.14-Q2 MID

```
print([(y := x) for x in range(3)], y)
```

2.3.14-Q3 HARD

```
print(any((m := v) > 2 for v in [1, 3, 5]), m)
```

2.3.15 Annotated assignment x: int = 5

Syntax name: type = value. Stored in __annotations__ at module/class level; not enforced. A bare "x: int" declares an annotation without creating the name.

Example 2.3.15.1: Annotations

```
x: int = 5
y: str
print(x)
print(__annotations__)
try:
    print(y)
except NameError as e:
    print("NameError:", e)
```

```
5
{'x': <class 'int'>, 'y': <class 'str'>}
NameError: name 'y' is not defined
```

Practice 2.3.15

2.3.15-Q1 BASIC

```
z: int = 3; print(z)
```

2.3.15-Q2 MID

```
a: int = "s"; print(a)
```

2.3.15-Q3 HARD

```
v: list[int]; print("v" in dir(), "v" in __annotations__)
```

2.3.16 Evaluation order puzzles

RHS is evaluated first, then targets are assigned left to right. When a later target depends on an earlier one, the result can surprise you.

Example 2.3.16.1: Order of binding

```
i = 0
l = [0, 0, 0]
i, l[i] = 1, 9
print(i, l)
```

```
1 [0, 9, 0]
```

Example 2.3.16.2: The same, written the other way round

```
i = 0
l = [0, 0, 0]
l[i], i = 9, 1
print(i, l)
```

```
1 [9, 0, 0]
```

Example 2.3.16.3: Fibonacci

```
a, b = 1, 1
a, b = b, a + b
print(a, b)
```

```
1 2
```

Practice 2.3.16

2.3.16-Q1 BASIC

```
a, b = 1, 2; a, b = b, a; print(a, b)
```

2.3.16-Q2 MID

```
i = 0; l = [0, 0]; i, l[i] = 1, 5; print(i, l)
```

2.3.16-Q3 HARD

```
l = [0, 0]; i = 0; l[i], i = 7, 1; print(i, l)
```

2.3.16-Q4 TRAP

```
a = [1, 2]; a[0], a[1] = a[1], a[0]; print(a)
```

2.3.17 Assignment inside loops and conditionals

Python has function-level scope, not block scope. Names created inside if/for/while remain after the block (if the block ran).

Example 2.3.17.1: Loop variable survives

```
for i in range(3):
    last = i * 10
print(i, last)
```

```
2 20
```

Example 2.3.17.2: Name defined in a branch that did not run

```
if False:
    z = 1
try:
    print(z)
except NameError as e:
    print("NameError:", e)
```

```
NameError: name 'z' is not defined
```

Example 2.3.17.3: Empty loop leaves the name unbound

```
for q in []:
    pass
print("q" in dir())
```

```
False
```

Practice 2.3.17

2.3.17-Q1 BASIC

```
for i in range(3): pass
print(i)
```

2.3.17-Q2 MID

```
for i in range(0): pass
print("i" in dir())
```

2.3.17-Q3 HARD

```
x = 10
for x in range(3): pass
print(x)
```

2.3.18 Assignment mini-lab

Fifteen puzzles. Predict first, then compare.

Practice 2.3.18

2.3.18-Q1 BASIC

```
a = b = 1; b += 1; print(a, b)
```

2.3.18-Q2 BASIC

```
x, y = 1, 2; x, y = y, x; print(x, y)
```

2.3.18-Q3 MID

```
a, *b = "hey"; print(a, b)
```

2.3.18-Q4 MID

```
x = [1]; y = x; x += [2]; print(y)
```

2.3.18-Q5 MID

```
a, (b, *c) = 1, (2, 3, 4); print(a, b, c)
```

2.3.18-Q6 MID

```
i = 0; i += i + 1; print(i)
```

2.3.18-Q7 HARD

```
a = [1, 2]; a[0], a[1] = a[1], a[0]; print(a)
```

2.3.18-Q8 HARD

```
i = 1; l = [0, 0, 0]; i, l[i] = 2, 5; print(i, l)
```

2.3.18-Q9 HARD

```
x = (y := 3) + y; print(x)
```

2.3.18-Q10 HARD

```
a = [[]] * 2; a[0].append(1); print(a)
```

2.3.18-Q11 TRAP

```
a = b = []; a += [1]; b = b + [2]; print(a, b)
```

2.3.18-Q12 TRAP

```
t = (1, 2); a, b = t; t = (b, a); print(a, b, t)
```

2.3.18-Q13 TRAP

```
x = 1; x = x if x > 1 else x + 10; print(x)
```

2.3.18-Q14 TRAP

```
a, b, *c = [1, 2]; print(c)
```

2.3.18-Q15 TRAP

```
n = 5; n -= n; n += 1; print(n)
```

Section 2.4 Scope (Variables Lifespan)

2.4.1 Local / Enclosing / Global / Built-in (LEGB)

When Python reads a name it searches four scopes in order: Local (current function), Enclosing (outer functions), Global (module), Built-in (len, print, ...). The first match wins. Assigning to a name inside a function makes it local to that function.

Example 2.4.1.1: One name at four levels

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)
    inner()
    print(x)
outer()
print(x)
```

```
local
enclosing
global
```

Example 2.4.1.2: Built-in is the last resort

```
print(len([1, 2]))
def f():
    return len
print(f() is len)
```

```
2
True
```

Example 2.4.1.3: Namespaces as dicts

```
x = 1
def f():
    y = 2
    return locals()
print(f())
print("x" in globals())
```

```
{'y': 2}
True
```

Practice 2.4.1

2.4.1-Q1 BASIC

```
x = 1
def f(): return x
print(f())
```

2.4.1-Q2 MID

```
x = 1
def f():
    x = 2
f()
print(x)
```

2.4.1-Q3 MID

```
def outer():
    v = "e"
    def inner(): return v
    return inner()
print(outer())
```

2.4.1-Q4 HARD

```
x = "g"
def f():
    print(x)
    x = "l"
try:
    f()
except UnboundLocalError:
    print("UnboundLocalError")
```

2.4.2 global keyword

Reading a global needs no declaration. Rebinding one inside a function does: "global x" tells Python the name refers to the module-level x. Mutating a global object (list.append) needs no declaration because no rebinding happens.

Example 2.4.2.1: Rebind needs global; mutate does not

```
count = 0
log = []
def bump():
    global count
    count += 1
    log.append(count)
bump(); bump()
print(count, log)
```

```
2 [1, 2]
```

Example 2.4.2.2: Without global

```
n = 0
def f():
    n = 99
f()
print(n)
```

```
0
```

Practice 2.4.2

2.4.2-Q1 BASIC

```
g = 1
def f():
    global g
    g = 2
f()
print(g)
```

2.4.2-Q2 MID

```
l = []
def f(): l.append(1)
f(); f(); print(l)
```

2.4.2-Q3 HARD

```
n = 0
def f():
    n += 1
try:
    f()
except UnboundLocalError:
    print("error")
```

2.4.3 nonlocal keyword

nonlocal x rebinds a variable of the nearest enclosing function (never the module). Used for closures that keep state.

Example 2.4.3.1: Counter closure

```
def make_counter():
    c = 0
    def inc():
        nonlocal c
        c += 1
        return c
    return inc
f = make_counter()
print(f(), f(), f())
g = make_counter()
print(g())
```

```
1 2 3
1
```

Example 2.4.3.2: nonlocal cannot target module level

```
try:
    compile("x = 1\ndef f():\n    nonlocal x\n", "<s>", "exec")
except SyntaxError as e:
    print("SyntaxError:", e.msg)
```

```
SyntaxError: no binding for nonlocal 'x' found
```

Practice 2.4.3

2.4.3-Q1 BASIC

```
def f():
    a = 1
    def g():
        nonlocal a
        a = 5
    g()
    return a
print(f())
```

2.4.3-Q2 MID

```
def f():
    a = 1
    def g():
        a = 5
    g()
    return a
print(f())
```

2.4.3-Q3 HARD

```
def mk():
    t = []
    def add(x):
        t.append(x)
        return len(t)
    return add
a = mk(); print(a(1), a(1), mk()(1))
```

2.4.4 Shadowing

A local name hides an outer name of the same spelling. Shadowing built-ins (list, sum) or standard modules (a file named random.py) is a frequent source of baffling errors.

Example 2.4.4.1: Local hides global

```
v = "outer"
def f():
    v = "inner"
    return v
print(f(), v)
```

```
inner outer
```

Example 2.4.4.2: Parameter shadows global

```
n = 10
def f(n): return n * 2
print(f(3), n)
```

```
6 10
```

Example 2.4.4.3: A file named like a module (simulated)

```
import sys, types
fake = types.ModuleType("random")
fake.__dict__["note"] = "I am your file, not the stdlib"
sys.modules["random"] = fake
import random
print(hasattr(random, "randint"), random.note)
```

```
False I am your file, not the stdlib
```

Practice 2.4.4

2.4.4-Q1 BASIC

```
x = 1
def f(x): return x + 1
print(f(5), x)
```

2.4.4-Q2 MID

```
len = 3
print(len)
del len
print(len([1, 2]))
```

2.4.4-Q3 HARD

```
def f():
    sum = 0
    for i in [1, 2]: sum += i
    return sum
print(f(), sum([5]))
```

2.4.5 UnboundLocalError pitfall

If a function assigns to a name ANYWHERE in its body, that name is local for the WHOLE function. Reading it before the assignment raises UnboundLocalError. Fix with global/nonlocal, or pass it as a parameter.

Example 2.4.5.1: The error

```
x = 10
def f():
    print(x)
    x = 20
try:
    f()
except UnboundLocalError as e:
    print("UnboundLocalError:", e)
```

```
UnboundLocalError: cannot access local variable 'x' where it is not associated with a value
```

Example 2.4.5.2: Three fixes

```
x = 10
def a():
    global x
    print(x); x = 20
def b(x):
    print(x); x = 20
    return x
def c():
    y = x
    print(y); y = 20
a(); print(b(x)); c()
```

```
10
20
20
20
```

Practice 2.4.5

2.4.5-Q1 BASIC

```
a = 1
def f():
    print(a)
f()
```

2.4.5-Q2 MID

```
a = 1
def f():
    a += 1
try:
    f()
except UnboundLocalError:
    print("UnboundLocalError")
```

2.4.5-Q3 HARD

```
a = 1
def f():
    if False:
        a = 2
    return a
try:
    f()
except UnboundLocalError:
    print("UnboundLocalError")
```

2.4.6 Module-level variables

Names assigned at the top level of a file live in the module's namespace (globals()). Importing runs the module once and caches it; importing again returns the same module and its state.

Example 2.4.6.1: Module state is shared by all importers

```
import types, sys
m = types.ModuleType("counter")
exec("count = 0\ndef bump():\n    global count\n    count += 1\n    return count", m.__dict__)
sys.modules["counter"] = m
import counter
counter.bump(); counter.bump()
import counter as c2
print(c2.count, c2 is counter)
```

```
2 True
```

Practice 2.4.6

2.4.6-Q1 BASIC

```
import math; print(math.pi > 3)
```

2.4.6-Q2 MID

```
import sys; import math as m1; import math as m2; print(m1 is m2)
```

2.4.6-Q3 HARD

```
import sys; print("math" in sys.modules or True)
```

2.4.7 __main__ scope (intro)

A script run directly is the module __main__ and its __name__ equals "__main__". When the same file is imported, __name__ is the module's file name. The guard if __name__ == "__main__": runs code only when executed directly.

Example 2.4.7.1: Simulating both situations

```
code = 'print("name is", __name__)\nif __name__ == "__main__":\n    print("running as script")'
exec(code, {"__name__": "__main__"})
exec(code, {"__name__": "mymodule"})
```

```
name is __main__
running as script
name is mymodule
```

Practice 2.4.7

2.4.7-Q1 BASIC

```
print(__name__)
```

2.4.7-Q2 MID

```
print("__main__" == __name__)
```

2.4.8 Block scope does not exist

if, for, while, with, try do not create scopes. Functions, lambdas, classes and comprehensions do.

Example 2.4.8.1: Block versus function

```
if True:
    inside_if = 1
print(inside_if)
def f():
    inside_f = 1
f()
print("inside_f" in dir())
```

```
1
False
```

Example 2.4.8.2: Comprehension has its own scope

```
r = [t for t in range(3)]
print("t" in dir())
```

```
False
```

Practice 2.4.8

2.4.8-Q1 BASIC

```
if True:
    a = 5
print(a)
```

2.4.8-Q2 MID

```
[k for k in range(3)]
print("k" in dir())
```

2.4.8-Q3 HARD

```
with open(__file__ if False else "/dev/null") as fh: name = fh.name
print(name, "fh" in dir())
```

2.4.9 Comprehension scope and the loop variable leak

In Python 3 comprehension variables do not leak. A walrus target inside a comprehension, however, binds in the enclosing scope. In a class body, comprehensions cannot see class-level names (except the outermost iterable).

Example 2.4.9.1: Leak and no-leak

```
x = "outer"
l = [x for x in range(3)]
print(x)
r = [(last := v) for v in range(3)]
print(last)
```

```
outer
2
```

Example 2.4.9.2: Class-body gotcha

```
class C:
    base = 10
    try:
        vals = [base + i for i in range(3)]
    except NameError as e:
        print("NameError:", e)
    vals2 = [i + 1 for i in range(base)]
print(C.vals2[:3])
```

```
NameError: name 'base' is not defined
[1, 2, 3]
```

Practice 2.4.9

2.4.9-Q1 BASIC

```
x = 5; [x for x in range(2)]; print(x)
```

2.4.9-Q2 MID

```
[w := i for i in range(3)]; print(w)
```

2.4.10 Closures: first look

A closure is an inner function that remembers variables from its enclosing scope. It captures the VARIABLE, not the value at definition time (late binding), which surprises people in loops.

Example 2.4.10.1: Late binding

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

```
[2, 2, 2]
```

Example 2.4.10.2: Fix with a default argument

```
fs = [lambda i=i: i for i in range(3)]
print([f() for f in fs])
```

```
[0, 1, 2]
```

Example 2.4.10.3: Fix with a factory

```
def make(i):
    return lambda: i
print([make(i)() for i in range(3)])
```

```
[0, 1, 2]
```

Example 2.4.10.4: Inspect the captured variable

```
def outer():
    v = 7
    return lambda: v
f = outer()
print(f.__closure__[0].cell_contents)
```

```
7
```

Practice 2.4.10

2.4.10-Q1 BASIC

```
f = (lambda x: lambda y: x + y)(1); print(f(2))
```

2.4.10-Q2 MID

```
fs = [lambda: n for n in "ab"]; print(fs[0]())
```

2.4.10-Q3 HARD

```
fs = []
for i in range(2):
    fs.append(lambda: i)
i = 9
print(fs[0](), fs[1]())
```

2.4.11 Class scope

A class body runs like a function whose namespace becomes the class attributes. Methods do not see class-level names as plain variables; use self.name or ClassName.name.

Example 2.4.11.1: Class attribute vs method lookup

```
class A:
    n = 5
    def get(self):
        try:
            return n
        except NameError:
            return "NameError, use self.n -> %d" % self.n
print(A().get())
```

```
NameError, use self.n -> 5
```

Example 2.4.11.2: Shared mutable class attribute

```
class B:
    items = []
a, b = B(), B()
a.items.append(1)
print(b.items, B.items)
```

```
[1] [1]
```

Practice 2.4.11

2.4.11-Q1 BASIC

```
class A:
    x = 1
print(A.x, A().x)
```

2.4.11-Q2 MID

```
class A:
    x = 1
a = A(); a.x = 2
print(A.x, a.x)
```

2.4.11-Q3 HARD

```
class A:
    l = []
a, b = A(), A(); a.l += [1]; print(b.l)
```

2.4.12 Exception variable scope

"except E as e" binds e only inside the except block; Python deletes it at the end to avoid reference cycles. Save it under another name if you need it later.

Example 2.4.12.1: Deleted after the block

```
try:
    1 / 0
except ZeroDivisionError as e:
    saved = e
print("e" in dir(), type(saved).__name__)
```

```
False ZeroDivisionError
```

Practice 2.4.12

2.4.12-Q1 BASIC

```
try:
    int("x")
except ValueError as err:
    pass
print("err" in dir())
```

2.4.12-Q2 MID

```
e = 5
try:
    1 / 0
except ZeroDivisionError as e:
    pass
print("e" in dir())
```

2.4.13 Namespaces and dir()/vars()/globals()/locals()

Every scope is a mapping from names to objects. globals() is the module dict; locals() is the function's local names (a snapshot you must not write to); vars(obj) is obj.__dict__; dir(obj) lists attribute names.

Example 2.4.13.1: Inspecting

```
a = 1
def f(p):
    q = 2
    return sorted(locals())
print(f(0))
print("a" in globals(), globals()["a"])
class K:
    z = 1
    def __init__(self): self.w = 2
print(vars(K()), "z" in dir(K()))
```

```
['p', 'q']
True 1
{'w': 2} True
```

Practice 2.4.13

2.4.13-Q1 BASIC

```
x = 1; print("x" in globals())
```

2.4.13-Q2 MID

```
def f(a, b): return sorted(locals())
print(f(1, 2))
```

2.4.13-Q3 HARD

```
def f():
    x = 1
    locals()["x"] = 99
    return x
print(f())
```

2.4.14 Global state in competitive programming

Local variables are faster than globals in CPython, so contest solutions put code inside main(). Deep recursion needs an explicit stack or a raised recursion limit.

Example 2.4.14.1: Timing the same loop (shape of the result, not exact numbers)

```
import timeit
g = 0
def with_global():
    global g
    for _ in range(100000): g += 1
def with_local():
    l = 0
    for _ in range(100000): l += 1
tg = timeit.timeit(with_global, number=20)
tl = timeit.timeit(with_local, number=20)
print("local faster:", tl < tg)
```

```
local faster: True
```

Practice 2.4.14

2.4.14-Q1 BASIC

```
import sys; print(sys.getrecursionlimit() >= 1000)
```

2.4.14-Q2 MID

```
def f(n): return n if n == 0 else f(n - 1)
try:
    f(5000)
except RecursionError:
    print("RecursionError")
```

2.4.15 Scope mini-lab

Twelve programs. Predict, then check.

Practice 2.4.15

2.4.15-Q1 BASIC

```
x = 1
def f(): return x + 1
print(f())
```

2.4.15-Q2 BASIC

```
def f():
    y = 1
f()
print("y" in dir())
```

2.4.15-Q3 MID

```
x = 1
def f():
    global x
    x = 5
f(); print(x)
```

2.4.15-Q4 MID

```
def f():
    v = 1
    def g(): return v + 1
    return g()
print(f())
```

2.4.15-Q5 MID

```
a = 5
def f(a=a): return a
a = 6
print(f())
```

2.4.15-Q6 HARD

```
def f():
    return [i for i in range(2)], "i" in dir()
print(f())
```

2.4.15-Q7 HARD

```
x = 0
def f():
    x = 1
    def g():
        nonlocal x
        x += 1
        return x
    return g()
print(f(), x)
```

2.4.15-Q8 HARD

```
def f():
    try:
        return z
    except NameError:
        return "NameError"
z = 1
print(f())
```

2.4.15-Q9 TRAP

```
def f(l=[]):
    l.append(1)
    return l
f(); print(f())
```

2.4.15-Q10 TRAP

```
fs = [lambda x=x: x for x in range(3)]
print([f() for f in fs])
```

2.4.15-Q11 TRAP

```
n = 1
def f():
    n = n + 1
try:
    f()
except Exception as e:
    print(type(e).__name__)
```

2.4.15-Q12 TRAP

```
def f():
    a = 1
    def g():
        return a
    a = 2
    return g()
print(f())
```

Section 2.5 Mutability and Identity

2.5.1 Mutable vs immutable (contrast table)

An immutable object cannot change its value after creation: int, float, complex, bool, str, bytes, tuple, frozenset, range, None. A mutable object can: list, dict, set, bytearray, and most user-defined instances. "Changing" an immutable value means creating a new object and rebinding the name.

Why it matters: dictionary keys and set members must be hashable (effectively immutable); function arguments of mutable types can be changed by the callee; immutable objects can be safely shared and interned.

Example 2.5.1.1: Proof table: try to modify each type

```
tests = [
 ("int",   lambda: (lambda x: x.__setattr__("a", 1))(5)),
 ("str",   lambda: "abc".__setitem__(0, "x")),
 ("tuple", lambda: (1, 2).__setitem__(0, 9)),
 ("bytes", lambda: b"ab".__setitem__(0, 1)),
 ("list",  lambda: [1, 2].__setitem__(0, 9)),
 ("dict",  lambda: {}.__setitem__("k", 1)),
 ("set",   lambda: {1}.add(2)),
 ("bytearray", lambda: bytearray(b"ab").__setitem__(0, 1)),
]
for name, f in tests:
    try:
        f(); print(f"{name:10} mutable")
    except (TypeError, AttributeError) as e:
        print(f"{name:10} immutable ({type(e).__name__})")
```

```
int        immutable (AttributeError)
str        immutable (AttributeError)
tuple      immutable (AttributeError)
bytes      immutable (AttributeError)
list       mutable
dict       mutable
set        mutable
bytearray  mutable
```

Example 2.5.1.2: Hashability follows mutability

```
for obj in [1, "a", (1, 2), frozenset({1}), [1], {1}, {"a": 1}]:
    try:
        hash(obj); h = "hashable"
    except TypeError:
        h = "unhashable"
    print(type(obj).__name__, h)
```

```
int hashable
str hashable
tuple hashable
frozenset hashable
list unhashable
set unhashable
dict unhashable
```

Practice 2.5.1

2.5.1-Q1 BASIC

```
s = "abc"
try:
    s[0] = "x"
except TypeError:
    print("immutable")
```

2.5.1-Q2 MID

```
print(hash((1, 2)) == hash((1, 2)))
```

2.5.1-Q3 MID

```
d = {}
try:
    d[[1]] = 1
except TypeError as e:
    print("TypeError")
```

2.5.1-Q4 HARD

```
t = (1, [2]); t[1].append(3); print(t)
```

2.5.2 id() behaviour

id is stable for the object's whole life and unique among live objects. After an object is freed its id may be reused by a new object. In CPython id is the address, but that is an implementation detail.

Example 2.5.2.1: Stable and reusable

```
a = [1]
i = id(a)
a.append(2)
print(id(a) == i)
class T: pass
x = id(T())
y = id(T())
print("ids of temporaries can repeat:", x == y)
```

```
True
ids of temporaries can repeat: True
```

Example 2.5.2.2: Equal values are not the same object (built at run time)

```
a = int("500")
b = int("500")
print(a == b, id(a) == id(b))
```

```
True False
```

Practice 2.5.2

2.5.2-Q1 BASIC

```
a = []; print(id(a) == id(a))
```

2.5.2-Q2 MID

```
a = [1]; b = a; a = None; print(b)
```

2.5.2-Q3 HARD

```
print(id([]) == id([]))
```

2.5.3 is vs == (contrast, big pitfall)

== asks "do these have equal values?" (calls __eq__). "is" asks "are these the very same object?" (compares ids). Use "is" only for None, True, False and sentinel objects. Comparing numbers or strings with "is" gives answers that depend on interpreter optimisations.

Example 2.5.3.1: Equal but not identical

```
a = [1, 2]; b = [1, 2]
print(a == b, a is b)
x = int("1000"); y = int("1000")
print(x == y, x is y)
```

```
True False
True False
```

Example 2.5.3.2: is with literals warns

```
import warnings
with warnings.catch_warnings(record=True) as w:
    warnings.simplefilter("always")
    compile("x is 1", "<s>", "eval")
    print(w[0].category.__name__)
```

```
SyntaxWarning
```

Example 2.5.3.3: Custom __eq__ can lie, is cannot

```
class Always:
    def __eq__(self, other): return True
o = Always()
print(o == None, o is None)
```

```
True False
```

Practice 2.5.3

2.5.3-Q1 BASIC

```
print([] == [], [] is [])
```

2.5.3-Q2 MID

```
a = (1, 2); b = (1, 2); print(a == b)
```

2.5.3-Q3 MID

```
print(None is None, 0 is None, 0 == None)
```

2.5.3-Q4 HARD

```
x = float("nan"); print(x == x, x is x)
```

2.5.3-Q5 TRAP

```
a = int("300"); b = int("300"); print(a is b, a == b)
```

2.5.4 Small integer interning (-5..256)

CPython pre-creates int objects for -5 to 256 and reuses them. So two names bound to such values at run time share one object. Outside the range, each computation usually creates a new object. Inside ONE compiled block (a script or one cell) identical constants are merged by the compiler, which can make 257 is 257 True. Use int("257") to escape that.

Example 2.5.4.1: Edges of the cache (built at run time to avoid constant folding)

```
for n in [-6, -5, 0, 256, 257, 1000]:
    a = int(str(n)); b = int(str(n))
    print(n, a is b)
```

```
-6 False
-5 True
0 True
256 True
257 False
1000 False
```

Example 2.5.4.2: Constant folding confuses the picture

```
a = 257
b = 257
print("same code block:", a is b)
```

```
same code block: True
```

Example 2.5.4.3: Not part of the language

```
import sys
print(sys.version_info[:2] >= (3, 8))
print("Never rely on 'is' for numbers; the range is a CPython detail.")
```

```
True
Never rely on 'is' for numbers; the range is a CPython detail.
```

Practice 2.5.4

2.5.4-Q1 BASIC

```
a = int("5"); b = int("5"); print(a is b)
```

2.5.4-Q2 MID

```
a = int("256"); b = int("256"); print(a is b)
```

2.5.4-Q3 MID

```
a = int("257"); b = int("257"); print(a is b)
```

2.5.4-Q4 HARD

```
x = 1000; y = 1000; print(x is y)
```

2.5.4-Q5 HARD

```
print(int("-5") is int("-5"), int("-6") is int("-6"))
```

2.5.5 String interning

CPython interns strings that look like identifiers (letters, digits, underscore) when they are compile-time constants, and you can intern any string with sys.intern. Equal strings built at run time are often different objects. Interning makes == faster (pointer check first) and saves memory when millions of equal strings occur.

Example 2.5.5.1: Compile-time vs run-time strings

```
a = "hello"
b = "hel" + "lo"
c = "".join(["hel", "lo"])
print(a is b, a is c, a == c)
```

```
True False True
```

Example 2.5.5.2: Constants inside one code block are shared, even with spaces

```
x = "hello world"; y = "hello world"
print(x is y)
```

```
True
```

Example 2.5.5.3: sys.intern forces sharing

```
import sys
c = "".join(["he", "llo wor", "ld"])
d = "".join(["hello", " ", "world"])
print(c is d)
print(sys.intern(c) is sys.intern(d))
```

```
False
True
```

Practice 2.5.5

2.5.5-Q1 BASIC

```
import sys; a = sys.intern("x y"); b = sys.intern("x " + "y"); print(a is b)
```

2.5.5-Q2 MID

```
a = "".join(["a", "b"]); b = "ab"; print(a == b, a is b)
```

2.5.5-Q3 HARD

```
import sys; a = "".join(["a b"]); print(sys.intern(a) is sys.intern("a b"))
```

2.5.6 Copy pitfalls (a = b for lists)

b = a never copies. To get a new list use a.copy(), a[:], list(a) or copy.copy(a). These are SHALLOW: the new list is a new object but its items are the same objects.

Example 2.5.6.1: Alias vs copies

```
a = [1, 2, 3]
alias = a
c1 = a.copy(); c2 = a[:]; c3 = list(a)
a.append(4)
print(alias, c1, c2, c3)
```

```
[1, 2, 3, 4] [1, 2, 3] [1, 2, 3] [1, 2, 3]
```

Example 2.5.6.2: The 2D list trap

```
bad = [[0] * 3] * 3
bad[0][0] = 1
print(bad)
good = [[0] * 3 for _ in range(3)]
good[0][0] = 1
print(good)
```

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Example 2.5.6.3: Default argument sharing

```
def add(x, bucket=[]):
    bucket.append(x)
    return bucket
print(add(1), add(2))
```

```
[1, 2] [1, 2]
```

Practice 2.5.6

2.5.6-Q1 BASIC

```
a = [1]; b = a.copy(); b.append(2); print(a)
```

2.5.6-Q2 MID

```
g = [[0]] * 2; g[0][0] = 5; print(g)
```

2.5.6-Q3 MID

```
g = [[0] for _ in range(2)]; g[0][0] = 5; print(g)
```

2.5.6-Q4 HARD

```
a = [[1], [2]]; b = a[:]; b[0].append(9); print(a)
```

2.5.6-Q5 TRAP

```
def f(x, l=[]):
    l.append(x)
    return l
f(1); f(2); print(f(3))
```

2.5.7 Shallow vs deep copy (preview)

copy.copy duplicates only the outer container. copy.deepcopy duplicates the whole object graph, keeping shared references shared and handling cycles.

Example 2.5.7.1: Shallow shares inner objects, deep does not

```
import copy
a = [[1], [2]]
s = copy.copy(a); d = copy.deepcopy(a)
a[0].append(99)
print(s, d)
print(s[0] is a[0], d[0] is a[0])
```

```
[[1, 99], [2]] [[1], [2]]
True False
```

Example 2.5.7.2: deepcopy preserves internal sharing

```
import copy
x = [0]
a = [x, x]
b = copy.deepcopy(a)
print(b[0] is b[1], b[0] is x)
```

```
True False
```

Practice 2.5.7

2.5.7-Q1 BASIC

```
import copy; a = [1, [2]]; b = copy.deepcopy(a); b[1].append(3); print(a)
```

2.5.7-Q2 MID

```
import copy; a = [1, [2]]; b = copy.copy(a); b[1].append(3); print(a)
```

2.5.7-Q3 HARD

```
import copy; a = []; a.append(a); b = copy.deepcopy(a); print(b[0] is b)
```

2.5.8 Hashability

An object is hashable if hash(obj) works and equal objects have equal hashes. Dictionary keys and set elements must be hashable. Immutable built-ins are hashable if their contents are. Defining __eq__ without

__hash__ makes a class unhashable.

Example 2.5.8.1: Rules in practice

```
print(hash(1) == hash(1.0) == hash(True))
d = {1: "int", 1.0: "float", True: "bool"}
print(d)
print({(1, 2): "ok"})
try:
    {(1, [2]): "bad"}
except TypeError as e:
    print("TypeError:", e)
```

```
True
{1: 'bool'}
{(1, 2): 'ok'}
TypeError: unhashable type: 'list'
```

Example 2.5.8.2: __eq__ without __hash__

```
class P:
    def __init__(self, v): self.v = v
    def __eq__(self, o): return self.v == o.v
try:
    {P(1)}
except TypeError as e:
    print("TypeError:", e)
```

```
TypeError: unhashable type: 'P'
```

Practice 2.5.8

2.5.8-Q1 BASIC

```
print({1, 1.0, True})
```

2.5.8-Q2 MID

```
print(hash("a") == hash("a"))
```

2.5.8-Q3 HARD

```
print(hash(-1) == hash(-2))
```

2.5.9 Immutability is shallow

A tuple cannot change which objects it holds, but if one of them is mutable that object can change. t[0] += [x] on a tuple element performs the in-place extend and then fails when storing back, so you get both an exception AND a changed value.

Example 2.5.9.1: Tuple holding a list

```
t = (1, [2, 3])
t[1].append(4)
print(t)
```

```
(1, [2, 3, 4])
```

Example 2.5.9.2: The famous += puzzle

```
t = (1, [2])
try:
    t[1] += [3]
except TypeError as e:
    print("TypeError:", e)
print(t)
```

```
TypeError: 'tuple' object does not support item assignment
(1, [2, 3])
```

Practice 2.5.9

2.5.9-Q1 BASIC

```
t = ([],); t[0].append(1); print(t)
```

2.5.9-Q2 MID

```
t = (1, 2); print(t + (3,), t)
```

2.5.9-Q3 HARD

```
t = ([0],); t[0] += [1] if False else []; print(t)
```

2.5.10 Mutation through function parameters

Python passes references (call by object reference). A function can mutate a mutable argument in place and the caller sees it. Rebinding the parameter has no effect on the caller.

Example 2.5.10.1: Mutation vs rebinding

```
def mutate(l): l.append(1)
def rebind(l): l = [1]
def aug(l): l += [1]
def plus(l): l = l + [1]
a = []; mutate(a); print(a)
a = []; rebind(a); print(a)
a = []; aug(a); print(a)
a = []; plus(a); print(a)
```

```
[1]
[]
[1]
[]
```

Practice 2.5.10

2.5.10-Q1 BASIC

```
def f(x): x = 5
v = 1; f(v); print(v)
```

2.5.10-Q2 MID

```
def f(d): d["k"] = 1
z = {}; f(z); print(z)
```

2.5.10-Q3 HARD

```
def f(l): l[:] = [0]
a = [1, 2]; f(a); print(a)
```

2.5.10-Q4 TRAP

```
def f(s): s += "!"
s = "hi"; f(s); print(s)
```

2.5.11 Defensive copying

A function that must not change its argument should copy it first, or accept an immutable type (tuple). Return copies when exposing internal state.

Example 2.5.11.1: Safe function

```
def normalized(values):
    out = list(values)
    out.sort()
    return out
data = [3, 1, 2]
print(normalized(data), data)
```

```
[1, 2, 3] [3, 1, 2]
```

Example 2.5.11.2: Freezing state

```
class Bag:
    def __init__(self): self._items = []
    def add(self, x): self._items.append(x)
    def items(self): return tuple(self._items)
b = Bag(); b.add(1)
print(b.items())
```

```
(1,)
```

Practice 2.5.11

2.5.11-Q1 BASIC

```
def f(l): return sorted(l)
a = [2, 1]; f(a); print(a)
```

2.5.11-Q2 MID

```
def f(l): l.sort(); return l
a = [2, 1]; f(a); print(a)
```

2.5.12 Immutable tricks and performance

Tuples are smaller and slightly faster to create than lists. String += in a loop is O(n^2) in theory (CPython sometimes optimises it); build lists and join. Immutable objects can be cached and shared.

Example 2.5.12.1: Sizes on this machine

```
import sys
print(sys.getsizeof((1, 2, 3)) < sys.getsizeof([1, 2, 3]))
print(sys.getsizeof([]) < sys.getsizeof([1, 2, 3, 4, 5]))
```

```
True
True
```

Example 2.5.12.2: Timing join vs +=

```
import timeit
def concat():
    s = ""
    for i in range(2000): s += "x"
    return s
def joined():
    return "".join(["x" for _ in range(2000)])
print(concat() == joined())
```

```
True
```

Practice 2.5.12

2.5.12-Q1 BASIC

```
print(len("".join(["a"] * 5)))
```

2.5.12-Q2 MID

```
import sys; print(sys.getsizeof(()) < sys.getsizeof([]))
```

2.5.13 Object identity in containers

Containers store references. Membership (in) and index() test identity first, then equality. NaN is the famous case: it is not equal to itself, but a list can still "contain" the same NaN object.

Example 2.5.13.1: Identity then equality

```
nan = float("nan")
print(nan == nan, nan in [nan], float("nan") in [nan])
a = [1]; b = [1]
print([a, a] == [a, b], [a, a][0] is [a, b][1])
```

```
False True False
True False
```

Practice 2.5.13

2.5.13-Q1 BASIC

```
print(1 in [1.0], True in [1])
```

2.5.13-Q2 MID

```
n = float("nan"); print([n].count(n), [float("nan")].count(float("nan")))
```

2.5.13-Q3 HARD

```
a = [1]; l = [a, [1]]; print(l.index([1]), l[0] is l[1])
```

2.5.14 Mutability mini-lab

Fifteen puzzles on aliasing, copying, mutation and tuples.

Practice 2.5.14

2.5.14-Q1 BASIC

```
a = [1]; b = a; b.append(2); print(a)
```

2.5.14-Q2 BASIC

```
s = "ab"; t = s; t += "c"; print(s)
```

2.5.14-Q3 MID

```
a = [1, 2]; b = a[:]; b[0] = 9; print(a)
```

2.5.14-Q4 MID

```
a = [[1]]; b = a[:]; b[0][0] = 9; print(a)
```

2.5.14-Q5 MID

```
t = (1, 2); u = t; u += (3,); print(t)
```

2.5.14-Q6 MID

```
d = {"a": [1]}; e = dict(d); e["a"].append(2); print(d)
```

2.5.14-Q7 HARD

```
import copy; d = {"a": [1]}; e = copy.deepcopy(d); e["a"].append(2); print(d)
```

2.5.14-Q8 HARD

```
a = [1]; b = [a, a]; b[0].append(2); print(b)
```

2.5.14-Q9 HARD

```
x = [1]; y = x; x = x * 2; print(y)
```

2.5.14-Q10 HARD

```
x = [1]; y = x; x *= 2; print(y)
```

2.5.14-Q11 TRAP

```
t = ([],); t[0] += [1] if True else None; print(t)
```

2.5.14-Q12 TRAP

```
def f(a=[]): a.append(1); return len(a)
print(f(), f(), f())
```

2.5.14-Q13 TRAP

```
a = {1: [0]}; b = a.copy(); b[2] = 5; print(a)
```

2.5.14-Q14 TRAP

```
s = {1, 2}; t = s; s = s | {3}; print(t)
```

2.5.14-Q15 TRAP

```
s = {1, 2}; t = s; s |= {3}; print(t)
```

Section 2.6 Memory Model Internals

2.6.1 Objects on the heap, names in namespaces

Objects live in a private heap managed by Python. A namespace is a dictionary mapping names to objects. A function call creates a new local namespace; a module has a global one. sys.getsizeof(obj) returns the size of the object itself, not of things it references.

Example 2.6.1.1: Shallow size vs total size

```
import sys
inner = list(range(1000))
outer = [inner, inner]
print(sys.getsizeof(outer) < sys.getsizeof(inner))
```

```
True
```

Example 2.6.1.2: Namespaces are dicts

```
x = 1
print(type(globals()).__name__, globals()["x"])
```

```
dict 1
```

Practice 2.6.1

2.6.1-Q1 BASIC

```
print(type(globals()).__name__)
```

2.6.1-Q2 MID

```
x = 5; globals()["y"] = x + 1; print(y)
```

2.6.2 Reference counting

Each object stores a count of references. Assigning, passing and storing in containers increment it; rebinding, del and container removal decrement it. At zero the object is freed at once. This is why files and objects are cleaned up promptly in CPython.

Example 2.6.2.1: Watching the count change

```
import sys
class Thing: pass
t = Thing()
r0 = sys.getrefcount(t)
a = t; b = [t]; c = {"k": t}
print(sys.getrefcount(t) - r0)
del a, b, c
print(sys.getrefcount(t) - r0)
```

```
3
0
```

Example 2.6.2.2: Immediate destruction

```
class Loud:
    def __del__(self): print("freed")
x = Loud()
print("before")
x = None
print("after")
```

```
before
freed
after
```

Practice 2.6.2

2.6.2-Q1 BASIC

```
import sys; t = object(); r = sys.getrefcount(t); a = t; print(sys.getrefcount(t) - r)
```

2.6.2-Q2 MID

```
class L:
    def __del__(self): print("gone")
x = L(); y = x; del x; print("still"); del y
```

2.6.3 Cyclic garbage collector

Objects that refer to each other keep each other's count above zero even if nothing else uses them. The gc module finds and frees such cycles in generations. gc.collect() runs a full collection.

Example 2.6.3.1: A cycle survives until collection

```
import gc, weakref
class Node: pass
a = Node(); b = Node(); a.other = b; b.other = a
w = weakref.ref(a)
del a, b
print("alive after del:", w() is not None)
gc.collect()
print("alive after gc.collect():", w() is not None)
print(gc.get_threshold())
```

```
alive after del: True
alive after gc.collect(): False
(700, 10, 10)
```

Practice 2.6.3

2.6.3-Q1 BASIC

```
import gc; print(gc.isenabled())
```

2.6.3-Q2 MID

```
import gc, weakref
class N: pass
a = N(); a.me = a
w = weakref.ref(a); del a; gc.collect(); print(w() is None)
```

2.6.4 Object header and size of basic objects

Every object carries a header (reference count and type pointer). Sizes depend on version and platform; the numbers below are from this machine's Python 3.12 (64-bit). Use them for relative comparisons only.

Example 2.6.4.1: Sizes (bytes)

```
import sys
for name, o in [("None", None), ("0", 0), ("1", 1), ("2**30", 2**30), ("2**60", 2**60), ("1.5",
    1.5), ("''", ""), ("'a'", "a"), ("[]", []), ("[1]", [1]), ("()", ()), ("{}", {}), ("set()",
    set())]:
    print(f"{name:8} {sys.getsizeof(o)}")
```

```
None     16
0        28
1        28
2**30    32
2**60    36
1.5      24
''       41
'a'      42
[]       56
[1]      64
()       40
{}       64
set()    216
```

Example 2.6.4.2: A list stores 8-byte references

```
import sys
for n in [0, 1, 2, 3, 4, 5, 9, 17]:
    print(n, sys.getsizeof([0] * n))
```

```
0 56
1 64
2 72
3 80
4 88
5 96
9 128
17 192
```

Practice 2.6.4

2.6.4-Q1 BASIC

```
import sys; print(sys.getsizeof(1) == sys.getsizeof(2))
```

2.6.4-Q2 MID

```
import sys; print(sys.getsizeof(2**100) > sys.getsizeof(2))
```

2.6.5 Integer storage in CPython

CPython stores an int as an array of 30-bit "digits" plus a sign, so ints have no fixed upper limit. Memory grows by 4 bytes per extra 30 bits. 2**100 is one object with four digits.

Example 2.6.5.1: Growth per 30 bits

```
import sys
base = sys.getsizeof(0)
for bits in [0, 30, 60, 90, 120]:
    n = 2 ** bits
    print(bits, sys.getsizeof(n))
print((2**100).bit_length(), 2**100)
```

```
0 28
30 32
60 36
90 40
120 44
101 1267650600228229401496703205376
```

Example 2.6.5.2: No overflow

```
print(2**200 + 1)
print(len(str(2**1000)))
```

```
160693804425899027554196209234116260252220299378279283530137 7
302
```

Practice 2.6.5

2.6.5-Q1 BASIC

```
print((2**64).bit_length())
```

2.6.5-Q2 MID

```
import sys; print(sys.getsizeof(2**59) == sys.getsizeof(2**30 + 1))
```

2.6.6 id reuse after deletion

Freed memory is reused. So id(a) == id(b) can be True for two DIFFERENT objects if the first died before the second was created. Never use id comparison to decide that two objects are "the same" unless both are alive.

Example 2.6.6.1: Dead object, reused address

```
class T: pass
a = id(T())
b = id(T())
print(a == b)
x = T(); i = id(x); del x
y = T()
print(id(y) == i)
```

```
True
True
```

Practice 2.6.6

2.6.6-Q1 BASIC

```
print(id([]) == id([]))
```

2.6.6-Q2 MID

```
a = [1]; b = [1]; print(id(a) == id(b))
```

2.6.7 Weak references and del

A weak reference does not keep an object alive. weakref.ref(obj)() returns the object or None after it dies. Useful for caches and observers.

Example 2.6.7.1: Cache that does not leak

```
import weakref
class Img: pass
cache = weakref.WeakValueDictionary()
i = Img(); cache["a"] = i
print(len(cache))
del i
print(len(cache))
```

```
1
0
```

Practice 2.6.7

2.6.7-Q1 BASIC

```
import weakref
class X: pass
x = X(); r = weakref.ref(x); print(r() is x)
```

2.6.7-Q2 MID

```
import weakref
try:
    weakref.ref([])
except TypeError as e:
    print("TypeError")
```

2.6.8 Memory profiling tools

sys.getsizeof is shallow. tracemalloc tracks allocations by line. For deep size, walk the object graph. Measure before you optimise.

Example 2.6.8.1: tracemalloc basics

```
import tracemalloc
tracemalloc.start()
data = [str(i) for i in range(10000)]
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
print(cur > 100_000, peak >= cur)
```

```
True True
```

Example 2.6.8.2: A simple deep size helper

```
import sys
def deep(o, seen=None):
    seen = seen or set()
    if id(o) in seen: return 0
    seen.add(id(o))
    s = sys.getsizeof(o)
    if isinstance(o, dict): s += sum(deep(k, seen) + deep(v, seen) for k, v in o.items())
    elif isinstance(o, (list, tuple, set, frozenset)): s += sum(deep(i, seen) for i in o)
    return s
print(deep([[1, 2], [1, 2]]) > sys.getsizeof([[1, 2], [1, 2]]))
```

```
True
```

Practice 2.6.8

2.6.8-Q1 BASIC

```
import tracemalloc; print(tracemalloc.is_tracing())
```

2.6.8-Q2 MID

```
import sys; print(sys.getsizeof([[1] * 100]) < sys.getsizeof([1] * 100))
```

Section 2.7 Annotations, Constants and Typing for Variables

2.7.1 Variable annotations

name: type = value attaches a hint stored in __annotations__ (module, class) and ignored at run time. Local variable annotations inside functions are not stored.

Example 2.7.1.1: Where annotations live

```
x: int = 1
y: "list[str]" = []
print(__annotations__)
class P:
    a: int
    b: str = "s"
print(P.__annotations__)
def f():
    z: int = 1
    return z
print(f.__annotations__, f())
```

```
{'x': <class 'int'>, 'y': 'list[str]'}
{'a': <class 'int'>, 'b': <class 'str'>}
{} 1
```

Practice 2.7.1

2.7.1-Q1 BASIC

```
v: int = 3; print(v, __annotations__)
```

2.7.1-Q2 MID

```
class C:
    a: int
print(hasattr(C, "a"), C.__annotations__)
```

2.7.2 typing.Final and constants

Final marks a name as not to be reassigned (checked by mypy/pyright). ClassVar marks class-level attributes. At run time they do nothing.

Example 2.7.2.1: Final is advisory

```
from typing import Final, ClassVar
LIMIT: Final = 10
LIMIT = 11
print(LIMIT)
class C:
    count: ClassVar[int] = 0
print(C.count)
```

```
11
0
```

Practice 2.7.2

2.7.2-Q1 BASIC

```
from typing import Final; X: Final = 1; print(X)
```

2.7.2-Q2 MID

```
from typing import Final; X: Final[int] = 1; X = 2; print(X)
```

2.7.3 Type aliases

An alias gives a name to a type expression. Use x: TypeAlias = ... (3.10+) or the type statement (3.12+). Aliases are for readers and type checkers.

Example 2.7.3.1: Three spellings (3.12 syntax shown)

```
from typing import TypeAlias
Vector: TypeAlias = list[float]
type Matrix = list[Vector]
def norm(v: Vector) -> float: return sum(x * x for x in v) ** 0.5
print(norm([3.0, 4.0]))
print(Matrix, Matrix.__value__)
```

```
5.0
Matrix list[list[float]]
```

Practice 2.7.3

2.7.3-Q1 BASIC

```
IntList = list[int]; print(IntList)
```

2.7.3-Q2 MID

```
type A = int | str; print(A.__value__)
```

2.7.4 Optional, Union and the | syntax

Optional[int] means int or None. From 3.10 write int | None. After "if x is not None" a type checker narrows the type. At run time unions are only hints.

Example 2.7.4.1: Equivalent forms

```
from typing import Optional, Union
print(Optional[int] == Union[int, None], (int | None) == Optional[int])
def f(x: int | None = None) -> int:
    return 0 if x is None else x
print(f(), f(5))
print(isinstance(3, int | str), isinstance(None, int | None))
```

```
True True
0 5
True True
```

Practice 2.7.4

2.7.4-Q1 BASIC

```
print(isinstance(1, int | str))
```

2.7.4-Q2 MID

```
print(int | None)
```

2.7.5 Runtime vs static typing

Static checkers (mypy, pyright) analyse annotations without running code; the interpreter ignores them. Gradual typing lets you add hints in steps. Python 3.14 evaluates annotations lazily (PEP 649). This machine runs 3.12, so annotations here are evaluated at definition time.

Example 2.7.5.1: Hints do not protect you at run time

```
def double(x: int) -> int:
    return x * 2
print(double(3), double("ab"), double([1]))
```

```
6 abab [1, 1]
```

Example 2.7.5.2: Reading hints with typing.get_type_hints

```
import typing
def f(a: int, b: "str" = "x") -> bool: ...
print(typing.get_type_hints(f))
```

```
{'a': <class 'int'>, 'b': <class 'str'>, 'return': <class 'bool'>}
```

Practice 2.7.5

2.7.5-Q1 BASIC

```
def f(x: int) -> int: return x
print(f("a"))
```

2.7.5-Q2 MID

```
def f(x: int): ...
print(f.__annotations__)
```

2.7.6 Mini-lab: annotate a script

Task: add annotations to five small functions, run mypy, fix three reported errors. Below is a typed starting file and the runtime result.

Example 2.7.6.1: Typed script

```
from typing import Iterable
def total(xs: Iterable[float]) -> float: return sum(xs)
def mean(xs: list[float]) -> float: return total(xs) / len(xs)
def clamp(x: float, lo: float = 0.0, hi: float = 1.0) -> float: return max(lo, min(x, hi))
def label(n: int | None) -> str: return "none" if n is None else str(n)
print(total([1, 2.5]), mean([1.0, 3.0]), clamp(5), label(None), label(3))
```

```
3.5 2.0 1.0 none 3
```

Practice 2.7.6

2.7.6-Q1 BASIC

```
def f(a: int, b: int = 2) -> int: return a * b
print(f(4))
```

2.7.6-Q2 MID

```
def g(x: "int | None") -> str: return "n" if x is None else "v"
print(g(None), g(1))
```

Section 2.8 Inspecting Variables

2.8.1 type, isinstance, id, dir, vars, repr

type shows the class, isinstance checks class relationship, id the identity, dir lists attributes, vars the __dict__, repr a developer-friendly string. Use repr (not str) when debugging: it shows quotes and escapes.

Example 2.8.1.1: One object, many probes

```
x = [1, "a"]
print(type(x), isinstance(x, list), isinstance(x, (list, tuple)))
print(repr("hi\n"), str("hi\n"))
print([n for n in dir(x) if not n.startswith("_")][:5])
class A:
    def __init__(self): self.v = 1
print(vars(A()), repr(A())[:12])
```

```
<class 'list'> True True
'hi\n' hi

['append', 'clear', 'copy', 'count', 'extend']
{'v': 1} <__main__.A
```

Practice 2.8.1

2.8.1-Q1 BASIC

```
print(repr("a"), str("a"))
```

2.8.1-Q2 MID

```
print(isinstance(True, int), type(True) == int)
```

2.8.1-Q3 HARD

```
print(sorted(vars(type("T", (), {"a": 1}))()))
```

2.8.2 globals() and locals() as dicts

globals() returns the live module dict, so writing to it creates names. locals() inside a function is a snapshot; changing it does not change local variables (and from 3.13 each call returns an independent snapshot).

Example 2.8.2.1: Writing

```
globals()["made"] = 42
print(made)
def f():
    a = 1
    snap = locals()
    snap["a"] = 99
    return a
print(f())
```

```
42
1
```

Practice 2.8.2

2.8.2-Q1 BASIC

```
globals()["k"] = 3; print(k)
```

2.8.2-Q2 MID

```
def f():
    x = 1
    d = locals()
    return list(d)
print(f())
```

2.8.3 inspect module first look

inspect gives signatures, source code, members and call stack information. Good for tooling and exploration, rarely needed in everyday code.

Example 2.8.3.1: Signature and members

```
import inspect
def f(a, b=2, *args, **kw): pass
print(inspect.signature(f))
print([n for n, _ in inspect.getmembers(int) if n == "bit_length"])
print(inspect.isfunction(f), inspect.isclass(int))
```

```
(a, b=2, *args, **kw)
['bit_length']
True True
```

Practice 2.8.3

2.8.3-Q1 BASIC

```
import inspect; print(inspect.isfunction(len), inspect.isbuiltin(len))
```

2.8.3-Q2 MID

```
import inspect; print(list(inspect.signature(lambda a, b=1: 0).parameters))
```

2.8.4 f"{x=}" debugging

f"{expr=}" prints the expression text, an equals sign and the repr of the value (3.8+). Combine with format specs.

Example 2.8.4.1: Self-documenting expressions

```
x = 3.14159; name = "ana"
print(f"{x=}")
print(f"{x=:.2f}")
print(f"{name=} {len(name)=}")
print(f"{x * 2 = }")
```

```
x=3.14159
x=3.14
name='ana' len(name)=3
x * 2 = 6.28318
```

Practice 2.8.4

2.8.4-Q1 BASIC

```
n = 5; print(f"{n=}")
```

2.8.4-Q2 MID

```
s = "a"; print(f"{s=!s}")
```

2.8.5 Trace tools: pdb, sys.settrace, tracemalloc

pdb (breakpoint()) pauses execution so you can inspect names. sys.settrace calls a function for each line event. tracemalloc shows where memory is allocated.

Example 2.8.5.1: A tiny line tracer

```
import sys
log = []
def tracer(frame, event, arg):
    if event == "line": log.append(frame.f_lineno)
    return tracer
def work():
    a = 1
    b = a + 1
    return b
sys.settrace(tracer); work(); sys.settrace(None)
print(len(log))
```

```
3
```

Practice 2.8.5

2.8.5-Q1 BASIC

```
print(callable(breakpoint))
```

2.8.5-Q2 MID

```
import sys; print(sys.gettrace())
```

2.8.6 Chapter 2 mini-lab: Variable Playground

Build a tool that prints type, a stable identity token, size and mutability for any expression, and detects aliasing between two names. Reference implementation:

Example 2.8.6.1: variable_playground.py (reference solution, run)

```
import sys
MUTABLE = (list, dict, set, bytearray)
def info(name, obj):
    return f"{name}: type={type(obj).__name__:9} size={sys.getsizeof(obj):4}
        mutable={isinstance(obj, MUTABLE)}"
def same(n1, o1, n2, o2):
    return f"{n1} is {n2}: {o1 is o2}   {n1} == {n2}: {o1 == o2}"
a = [1, 2]; b = a; c = [1, 2]; d = (1, 2); s = "hi"
for n, o in [("a", a), ("c", c), ("d", d), ("s", s)]:
    print(info(n, o))
print(same("a", a, "b", b))
print(same("a", a, "c", c))
```

```
a: type=list      size=  72 mutable=True
c: type=list      size=  72 mutable=True
d: type=tuple     size=  56 mutable=False
s: type=str       size=  43 mutable=False
a is b: True   a == b: True
a is c: False   a == c: True
```

Practice 2.8.6

2.8.6-Q1 BASIC

```
a = [1]; b = a; print(a is b)
```

2.8.6-Q2 MID

```
a = (1, 2); b = (1, 2); print(a == b)
```

2.8.6-Q3 HARD

```
a = "ab"; b = "a" + "b"; print(a == b)
```

Chapter 2 Summary: The 30 Rules

1. A variable is a name bound to an object, never a box holding a value.
2. Assignment evaluates the right side first, then binds the name.
3. Assignment never copies an object.
4. Types belong to objects, not names.
5. Python is dynamically and strongly typed: no silent str/int mixing.
6. Type hints are not enforced at run time.
7. id() is identity; type() is class; value may change only for mutable objects.
8. Rebinding moves an arrow; mutation changes the object.
9. a = b = [] creates aliases.
10. Use list.copy(), a[:], or copy.deepcopy to copy.
11. [[0]*3]*3 repeats one row three times.
12. Tuple unpacking needs matching counts unless one starred target is used.
13. The starred target always collects a list.
14. a, b = b, a evaluates the right tuple before binding.
15. x += y mutates mutable objects in place and rebinds immutable ones.
16. Chained comparison a < b < c evaluates b once.
17. del removes a name, not necessarily the object.
18. None is a singleton: test with is None.
19. Scope is function-level; blocks do not create scope.
20. LEGB: Local, Enclosing, Global, Built-in.
21. Assigning anywhere in a function makes the name local there.
22. global/nonlocal are needed to rebind, not to read or mutate.
23. Closures capture variables, not values (late binding).
24. Do not shadow built-ins (list, id, sum, max, input).
25. is is for None/True/False/sentinels; == is for values.
26. Never rely on small-int or string interning.
27. Immutability is shallow: a tuple can hold a mutable list.
28. Dict keys and set members must be hashable.
29. Mutable default arguments are shared across calls.
30. Reference counting frees objects at zero; the cycle collector handles cycles.

Error Message Decoder

Message	Usual cause	Fix
NameError: name 'x' is not defined	Used before assignment, typo, or wrong scope	Assign first; check spelling and scope
UnboundLocalError: cannot access local variable	Name assigned somewhere in the function but read before	Pass as parameter, or declare global/nonlocal

Message	Usual cause	Fix
TypeError: 'str' object does not support item assignment	Tried to mutate an immutable object	Build a new object and rebind
TypeError: unhashable type: 'list'	Mutable object used as dict key / set member	Use a tuple or frozenset
TypeError: 'int' object is not callable	A built-in or function name was shadowed	del the variable; rename
ValueError: too many values to unpack	Target count differs from iterable length	Match counts or use a starred target
ValueError: not enough values to unpack	Same as above, fewer values	Same
SyntaxError: cannot assign to literal	Left side of = is not a valid target	Use a name, item, attribute, or slice
SyntaxError: invalid syntax (keyword as name)	Reserved word used as identifier	Rename (class_)
RuntimeError: dictionary changed size during iteration	Mutated a dict while looping over it	Loop over list(d) or build a new dict
RecursionError	Too many nested calls	Iterate or raise the limit carefully
SyntaxWarning: 'is' with a literal	Used is to compare with a number/string	Use ==

Final Exam (45 mixed HARD and TRAP questions)

One question taken from each of 45 topics. Predict all outputs on paper, then check against the answer key. Score: 40+ mastery, 32-39 solid, below 32 revisit Sections 2.3 to 2.5.

E1 (from 2.1.1)

```
a = "hi"; b = a; b += "!"; print(a, b)
```

E2 (from 2.1.2)

```
a = [1]; b = [a, a]; a = [2]; print(b)
```

E3 (from 2.1.3)

```
print(type(type(5)))
```

E4 (from 2.1.4)

```
def f(l): l[:] = [7]
x = [1, 2]; f(x); print(x)
```

E5 (from 2.1.5)

```
def f(a, b): return a + b
print(f([1], [2]), f((1,), (2,)))
```

E6 (from 2.1.6)

```
print("5" + 5)
```

E7 (from 2.1.7)

```
x = 5; i = id(x); x += 1; print(id(x) == i)
```

E8 (from 2.1.8)

```
print(isinstance(int, type), isinstance(type, object), isinstance(object, type))
```

E9 (from 2.1.9)

```
try:
    print(w)
except NameError:
    print("no w")
w = 3
print(w)
```

E10 (from 2.1.10)

```
a = b = [1]; a = a + [2]; print(b)
```

E11 (from 2.1.11)

```
a = [1]; b = a; a *= 2; print(b)
```

E12 (from 2.1.12)

```
import weakref
class B: pass
b = B(); w = weakref.ref(b); del b; print(w() is None)
```

E13 (from 2.1.13)

```
a = [1]; b = a; a, b = b, a + [9]; print(a, b)
```

E14 (from 2.2.1)

```
import keyword; print(keyword.iskeyword("match"), "match".isidentifier())
```

E15 (from 2.2.2)

```
x = 1
X = x
x = 2
print(X)
```

E16 (from 2.2.3)

```
print("CapWords".isidentifier(), "CapWords"[0].isupper())
```

E17 (from 2.2.4)

```
import math; math.pi = 3; print(math.pi)
```

E18 (from 2.2.5)

```
import keyword; print("_" in keyword.softkwlist, "match" in keyword.softkwlist)
```

E19 (from 2.2.6)

```
print(5 .__eq__(5.0), 5 .__eq__("5"))
```

E20 (from 2.2.7)

```
class A:
    __v = 1
class B(A): pass
print(hasattr(B, "__v"), hasattr(B, "_A__v"))
```

E21 (from 2.2.8)

```
id = 5
print(type(id).__name__)
del id
print(type(id).__name__)
```

E22 (from 2.2.10)

```
for _ in range(2): pass
print(_)
```

E23 (from 2.2.11)

```
import unicodedata; print(unicodedata.normalize("NFKC", "ﬁ"))
```

E24 (from 2.3.1)

```
x = 3; y = (x := x + 1) * 2; print(x, y)
```

E25 (from 2.3.2)

```
a = b = {"k": 1}; b["k"] = 2; print(a)
```

E26 (from 2.3.3)

```
a, b = range(2, 4); print(a * b)
```

E27 (from 2.3.4)

```
a = 1; b = 2; a = b; b = a; print(a, b)
```

E28 (from 2.3.5)

```
a, *b, c = range(5); print(a, b, c)
```

E29 (from 2.3.6)

```
t = ([1], 2)
try:
    t[0] += [9]
except TypeError as e:
    print("TypeError")
print(t)
```

E30 (from 2.3.7)

```
print(False == False in [False])
```

E31 (from 2.3.8)

```
x = [1, 2]; y = x.append(3); print(y)
```

E32 (from 2.3.9)

```
l = [1, 2, 3]; del l[:]; print(l)
```

E33 (from 2.3.10)

```
print(None == False, not None)
```

E34 (from 2.3.11)

```
(a, *b), c = ("xyz", 9); print(a, b, c)
```

E35 (from 2.3.12)

```
print([*"hi", *(1, 2)], (*[1], *[2]))
```

E36 (from 2.3.13)

```
l = list(range(6)); l[::2] = ["a", "b", "c"]; print(l)
```

E37 (from 2.3.14)

```
print(any((m := v) > 2 for v in [1, 3, 5]), m)
```

E38 (from 2.3.15)

```
v: list[int]; print("v" in dir(), "v" in __annotations__)
```

E39 (from 2.3.16)

```
a = [1, 2]; a[0], a[1] = a[1], a[0]; print(a)
```

E40 (from 2.3.17)

```
x = 10
for x in range(3): pass
print(x)
```

E41 (from 2.3.18)

```
n = 5; n -= n; n += 1; print(n)
```

E42 (from 2.4.1)

```
x = "g"
def f():
    print(x)
    x = "l"
try:
    f()
except UnboundLocalError:
    print("UnboundLocalError")
```

E43 (from 2.4.2)

```
n = 0
def f():
    n += 1
try:
    f()
except UnboundLocalError:
    print("error")
```

E44 (from 2.4.3)

```
def mk():
    t = []
    def add(x):
        t.append(x)
        return len(t)
    return add
a = mk(); print(a(1), a(1), mk()(1))
```

E45 (from 2.4.4)

```
def f():
    sum = 0
    for i in [1, 2]: sum += i
    return sum
print(f(), sum([5]))
```

Answer Key (outputs from running the code)

Cover this section while you predict. Each answer is the real output produced by running the question.

2.1.1 What is a variable (name binding, not a box)

Q1 `5`

Q2 `[1, 2]`

Q3 `[1] [1, 2]`

Q4 `[1, 2] [1, 2]`

Q5 `hi hi!`

2.1.2 Variable as reference to an object

Q1 `True False`

Q2 `[[0, 9], [0, 9]]`

Q3 `[[0, 9], [0]]`

Q4 `[[1], [1]]`

2.1.3 id() and type(): first tools

Q1 `True`

Q2 `<class 'int'> <class 'float'> <class 'str'> <class 'list'>`

Q3 `True False`

Q4 `<class 'type'>`

2.1.4 Names vs values

Q1 `1`

Q2 `[1]`

Q3 `[]`

Q4 `[7]`

2.1.5 Dynamic typing (contrast with static typing)

Q1 `a`

Q2 `ababab ababab`

Q3 `2.5`

Q4 `[1, 2] (1, 2)`

2.1.6 Strong typing (Python is strongly + dynamically typed)

Q1 `aaa`

Q2 `2`

Q3 `ababc`

Q4 `TypeError: can only concatenate str (not "int") to str`

2.1.7 Objects: identity, type, value

Q1 `True`

Q2 `(1, 2, 3) False`

Q3 `False`

2.1.8 Everything is an object: first proof

Q1 `8`

Q2 `HI`

Q3 `builtin_function_or_method`

Q4 `True True True`

2.1.9 Variables are not declared

Q1 `False`

Q2 `NameError`

Q3
```
no w
    3
```

2.1.10 Multiple names for the same object

Q1 `[1]`

Q2 `1 0`

Q3 `[]`

Q4 `[[0, 1]]`

Q5 `[1]`

2.1.11 Rebinding does not affect other names

Q1 `1`

Q2 `[1]`

Q3 `[1, 2]`

Q4 `[1, 1]`

2.1.12 Variable lifetime and reference counting intro

Q1 `1`

Q2 `[1]`

Q3 `True`

2.1.13 Mental-model exercises (mini-lab)

Q1 `4 3`

Q2 `[1, 2] [1, 2]`

Q3 `[1, 2] [1, 2, 3]`

Q4 `(1, [2, 3])`

Q5 `xy x`

Q6 `[[1, 2], [1, 2], [1, 2]]`

Q7 `[[1, 2], [1, 2], [1, 2]]`

Q8 `[1, 2] [1, 3]`

Q9 `[0, 2] [0, 1] [0, 2]`

Q10 `[1] [1, 9]`

2.2.1 Valid identifier rules

Q1 `True False`

Q2 `True False`

Q3 `False True`

Q4 `False True`

2.2.2 Case sensitivity

Q1 `3`

Q2 `5 6`

Q3 `1`

2.2.3 snake_case convention (PEP 8)

Q1 `True False`

Q2 `True`

Q3 `True True`

2.2.4 Constant naming (UPPER_CASE, Final)

Q1 `3`

Q2 `2`

Q3 `3`

2.2.5 Reserved keywords as names (error)

Q1 `True`

Q2 `False`

Q3 `SyntaxError`

Q4 `True True`

2.2.6 Dunder names (__x__): intro

Q1 `8`

Q2 `3`

Q3 `NotImplemented NotImplemented`

2.2.7 Private-ish naming (_x, __x, __x__)

Q1 `1`

Q2 `['_A__v']`

Q3 `False True`

2.2.8 Beginner pitfalls: shadowing built-ins (list, id, input)

Q1 `3`

Q2 `shadowed`

Q3
```
int
    builtin_function_or_method
```

2.2.9 Naming for readability

Q1 `False`

Q2 `30000`

2.2.10 Throwaway name _

Q1 `1`

Q2 `3`

Q3 `1`

2.2.11 Unicode names and normalisation

Q1 `True`

Q2 `True`

Q3 `fi`

2.2.12 Naming in competitive programming

Q1 `[1, 1, 1]`

Q2 `2`

2.3.1 Simple assignment

Q1 `4`

Q2 `4`

Q3 `4 8`

2.3.2 Multiple assignment: a = b = c = 0

Q1 `5`

Q2 `[1, 2]`

Q3 `[1]`

Q4 `[0, 1] [0, 1]`

Q5 `{'k': 2}`

2.3.3 Tuple unpacking: a, b = 1, 2

Q1 `y`

Q2 `3`

Q3 `ValueError: not enough values to unpack (expected 2, got 1)`

Q4 `6`

2.3.4 Swap idiom: a, b = b, a

Q1 `4 3`

Q2 `3 1 2`

Q3 `2 3`

Q4 `2 2`

2.3.5 Starred unpacking: a, *b = [1,2,3]

Q1 `[2, 3]`

Q2 `['x', 'y'] z`

Q3 `[]`

Q4 `0 [1, 2, 3] 4`

2.3.6 Augmented assignment (+=, -=, *=, //=, **=, &=, |=, ^=, >>=, <<=, @=)

Q1 `14`

Q2 `[1, 2]`

Q3 `{1, 2}`

Q4 `[1]`

Q5 `TypeError`
`([1, 9], 2)`

2.3.7 Chained comparison: 1 < x < 10

Q1 `True`

Q2 `False`

Q3 `1`

Q4 `True`

Q5 `True`

2.3.8 Reassignment vs mutation

Q1 `[1, 2]`

Q2 `None [1, 3]`

Q3 `[1]`

Q4 `[1, 2, 3] [1, 2]`

Q5 `None`

2.3.9 del keyword

Q1 `[1, 3]`

Q2 `[1, 3, 5]`

Q3 `[1]`

Q4 `[]`

2.3.10 None as default

Q1 `True`

Q2 `False True`

Q3 `False True`

2.3.11 Nested unpacking

Q1 `6`

Q2 `a 1`

Q3 `x ['y', 'z'] 9`

2.3.12 Unpacking in function calls and literals

Q1 `[0, 1, 2]`

Q2 `{'a': 2}`

Q3 `['h', 'i', 1, 2] (1, 2)`

2.3.13 Assignment targets that are not names

Q1 `[1, 7, 8, 3]`

Q2 `[]`

Q3 `['a', 1, 'b', 3, 'c', 5]`

2.3.14 Walrus operator := first look

Q1 `5 4`

Q2 `[0, 1, 2] 2`

Q3 `True 3`

2.3.15 Annotated assignment x: int = 5

Q1 `3`

Q2 `s`

Q3 `False True`

2.3.16 Evaluation order puzzles

Q1 `2 1`

Q2 `1 [0, 5]`

Q3 `1 [7, 0]`

Q4 `[2, 1]`

2.3.17 Assignment inside loops and conditionals

Q1 `2`

Q2 `False`

Q3 `2`

2.3.18 Assignment mini-lab

Q1 `1 2`

Q2 `2 1`

Q3 `h ['e', 'y']`

Q4 `[1, 2]`

Q5 `1 2 [3, 4]`

Q6 `1`

Q7 `[2, 1]`

Q8 `2 [0, 0, 5]`

Q9 `6`

Q10 `[[1], [1]]`

Q11 `[1] [1, 2]`

Q12 `1 2 (2, 1)`

Q13 `11`

Q14 `[]`

Q15 `1`

2.4.1 Local / Enclosing / Global / Built-in (LEGB)

Q1 `1`

Q2 `1`

Q3 `e`

Q4 `UnboundLocalError`

2.4.2 global keyword

Q1 `2`

Q2 `[1, 1]`

Q3 `error`

2.4.3 nonlocal keyword

Q1 `5`

Q2 `1`

Q3 `1 2 1`

2.4.4 Shadowing

Q1 `6 1`

Q2 `3`
`2`

Q3 `3 5`

2.4.5 UnboundLocalError pitfall

Q1 `1`

Q2 `UnboundLocalError`

Q3 `UnboundLocalError`

2.4.6 Module-level variables

Q1 `True`

Q2 `True`

Q3 `True`

2.4.7 __main__ scope (intro)

Q1 `__main__`

Q2 `True`

2.4.8 Block scope does not exist

Q1 `5`

Q2 `False`

Q3 `/dev/null True`

2.4.9 Comprehension scope and the loop variable leak

Q1 `5`

Q2 `2`

2.4.10 Closures: first look

Q1 `3`

Q2 `b`

Q3 `9 9`

2.4.11 Class scope

Q1 `1 1`

Q2 `1 2`

Q3 `[1]`

2.4.12 Exception variable scope

Q1 `False`

Q2 `False`

2.4.13 Namespaces and dir()/vars()/globals()/locals()

Q1 `True`

Q2 `['a', 'b']`

Q3 `1`

2.4.14 Global state in competitive programming

Q1 `True`

Q2 `RecursionError`

2.4.15 Scope mini-lab

Q1 `2`

Q2 `False`

Q3 `5`

Q4 `2`

Q5 `5`

Q6 `([0, 1], False)`

Q7 `2 0`

Q8 `1`

Q9 `[1, 1]`

Q10 `[0, 1, 2]`

Q11 `UnboundLocalError`

Q12 `2`

2.5.1 Mutable vs immutable (contrast table)

Q1 `immutable`

Q2 `True`

Q3 `TypeError`

Q4 `(1, [2, 3])`

2.5.2 id() behaviour

Q1 `True`

Q2 `[1]`

Q3 `True`

2.5.3 is vs == (contrast, big pitfall)

Q1 `True False`

Q2 `True`

Q3 `True False False`

Q4 `False True`

Q5 `False True`

2.5.4 Small integer interning (-5..256)

Q1 `True`

Q2 `True`

Q3 `False`

Q4 `True`

Q5 `True False`

2.5.5 String interning

Q1 `True`

Q2 `True False`

Q3 `True`

2.5.6 Copy pitfalls (a = b for lists)

Q1 `[1]`

Q2 `[[5], [5]]`

Q3 `[[5], [0]]`

Q4 `[[1, 9], [2]]`

Q5 `[1, 2, 3]`

2.5.7 Shallow vs deep copy (preview)

Q1 `[1, [2]]`

Q2 `[1, [2, 3]]`

Q3 `True`

2.5.8 Hashability

Q1 `{1}`

Q2 `True`

Q3 `True`

2.5.9 Immutability is shallow

Q1 `([1],)`

Q2 `(1, 2, 3) (1, 2)`

Q3 `TypeError: 'tuple' object does not support item assignment`

2.5.10 Mutation through function parameters

Q1 `1`

Q2 `{'k': 1}`

Q3 `[0]`

Q4 `hi`

2.5.11 Defensive copying

Q1 `[2, 1]`

Q2 `[1, 2]`

2.5.12 Immutable tricks and performance

Q1 `5`

Q2 `True`

2.5.13 Object identity in containers

Q1 `True True`

Q2 `1 0`

Q3 `0 False`

2.5.14 Mutability mini-lab

Q1 `[1, 2]`

Q2 `ab`

Q3 `[1, 2]`

Q4 `[[9]]`

Q5 `(1, 2)`

Q6 `{'a': [1, 2]}`

Q7 `{'a': [1]}`

Q8 `[[1, 2], [1, 2]]`
Q9 `[1]`
Q10 `[1, 1]`
Q11 `TypeError: 'tuple' object does not support item assignment`
Q12 `1 2 3`
Q13 `{1: [0]}`
Q14 `{1, 2}`
Q15 `{1, 2, 3}`

2.6.1 Objects on the heap, names in namespaces

Q1 `dict`
Q2 `6`

2.6.2 Reference counting

Q1 `1`
Q2
```
still
    gone
```

2.6.3 Cyclic garbage collector

Q1 `True`
Q2 `True`

2.6.4 Object header and size of basic objects

Q1 `True`
Q2 `True`

2.6.5 Integer storage in CPython

Q1 `65`
Q2 `True`

2.6.6 id reuse after deletion

Q1 `True`
Q2 `False`

2.6.7 Weak references and del

Q1 `True`
Q2 `TypeError`

2.6.8 Memory profiling tools

Q1 `False`
Q2 `True`

2.7.1 Variable annotations

Q1 `3 {'v': <class 'int'>}`
Q2 `False {'a': <class 'int'>}`

2.7.2 typing.Final and constants

Q1 `1`

Q2 `2`

2.7.3 Type aliases

Q1 `list[int]`

Q2 `int | str`

2.7.4 Optional, Union and the | syntax

Q1 `True`

Q2 `int | None`

2.7.5 Runtime vs static typing

Q1 `a`

Q2 `{'x': <class 'int'>}`

2.7.6 Mini-lab: annotate a script

Q1 `8`

Q2 `n v`

2.8.1 type, isinstance, id, dir, vars, repr

Q1 `'a' a`

Q2 `True False`

Q3 `TypeError: 'mappingproxy' object is not callable`

2.8.2 globals() and locals() as dicts

Q1 `3`

Q2 `['x']`

2.8.3 inspect module first look

Q1 `False True`

Q2 `['a', 'b']`

2.8.4 f"{x=}" debugging

Q1 `n=5`

Q2 `s=a`

2.8.5 Trace tools: pdb, sys.settrace, tracemalloc

Q1 `True`

Q2 `None`

2.8.6 Chapter 2 mini-lab: Variable Playground

Q1 `True`

Q2 `True`

Q3 `True`

Final exam answers

E1 `hi hi!`

E2 `[[1], [1]]`

E3 `<class 'type'>`

E4 `[7]`

E5 `[1, 2] (1, 2)`

E6 `TypeError: can only concatenate str (not "int") to str`

E7 `False`

E8 `True True True`

E9
```
no w
3
```

E10 `[1]`

E11 `[1, 1]`

E12 `True`

E13 `[1] [1, 9]`

E14 `False True`

E15 `1`

E16 `True True`

E17 `3`

E18 `True True`

E19 `NotImplemented NotImplemented`

E20 `False True`

E21
```
int
builtin_function_or_method
```

E22 `1`

E23 `fi`

E24 `4 8`

E25 `{'k': 2}`

E26 `6`

E27 `2 2`

E28 `0 [1, 2, 3] 4`

E29
```
TypeError
([1, 9], 2)
```

E30 `True`

E31 `None`

E32 `[]`

E33 `False True`

E34 `x ['y', 'z'] 9`

E35 `['h', 'i', 1, 2] (1, 2)`

E36 `['a', 1, 'b', 3, 'c', 5]`

E37 `True 3`

E38 `False True`

E39 `[2, 1]`

E40 `2`

E41 `1`

E42 `UnboundLocalError`

E43 `error`

E44 `1 2 1`

E45 `3 5`